Dynamic In-Context Guidance for Guided Conversations
Tool-Authored Elicitations as a Reliability Lever

Two Controlled Studies on a Constrained Support-Intake Benchmark

Ryan Porter Matthew Belchak

April 24, 2026

Abstract

Teams building *guided conversations* in production—agentic workflows that must follow a prescribed script, in a prescribed order, with a strict definition of *done*, of the kind common in commerce, support, claims, and compliance—typically reach for one of two architectures. Either the agent is configured with one long upfront system prompt that enumerates the entire workflow, or the system maintains several agent configurations (each with its own system prompt) and routes each turn to one of them. Both treat the system prompt as the primary unit of control, and both have proven brittle in practice on workflows with hard procedural constraints. We argue for a third option: a single conversational role whose context is engineered *dynamically and just-in-time*, primarily by tools that author short, schema-typed elicitations at the moment they are needed. The Model Context Protocol's elicitation pattern is the obvious template, and we use it as our model. We support the argument with two controlled studies on a constrained support-intake benchmark with three personas and a fast role-playing simulator. **Experiment one** compares an upfront-instructions baseline with a guided arm whose structured capture is mediated by a single inline tool, `collect_field(name, value?)`, that returns an MCP-style elicitation payload on the first call and validates the user's reply on the second; both arms share task, validators, simulator, and model, and we report strict success, completion, and infra failure with Wilson 95% intervals at $N = 100$ per persona per arm in two simulator client modes (*ideal*, *non_ideal*). The guided arm reaches 99.0% strict success at *ideal* (vs. 53.3% baseline; equal-weighted means across personas) and 71.7% at *non_ideal* (vs. 56.7%); completion goes from 47–80% to a uniform 100% under both modes. **Experiment two** replaces the form-filling intervention with a top-level *orchestrator* tool that authors the conversation's phase guidance, and compares a strict ordering against a flexible ordering under an *impatient* simulator client mode that hangs up if the agent repeatedly deflects the user's preferred topic;

both orchestrators perform the same form-filling work via nested sub-tools, isolating the orchestration trade-off. Both orchestrators reach 83–89% strict success vs. the upfront baseline's 40% (equal-weighted across personas, $N = 100$ each), and the rigid-vs-loose comparison is interestingly persona-shaped: loose dominates on the persona whose preferred topic flows naturally into form-filling, ties on the persona whose required fields cover the topic, and concedes a 16% hung-up rate on the rambling persona where each topic-acknowledgment turn eats more of the patience budget. Across both studies we frame the question as one of *control architecture*, not model competence: how much of an agent's reliability on a constrained workflow is shaped by where the procedural detail lives—one upfront blob, one of several upfront blobs, or a sequence of small, just-in-time, tool-authored messages.

1 Introduction

LLM agents are increasingly asked to drive *guided conversations*: regulated support intake, claims and onboarding flows, triage scripts, compliance interviews, and other workflows where conversational fluency is necessary but not sufficient. These workflows impose ordering rules, branch-specific requirements, mandatory disclosures, and a strict definition of *done*. They cannot be left to a freely improvising agent.

Two architectures dominate today's attempts to add this kind of guidance on top of a chat-completions-style runtime. The first is the *long upfront prompt*: every step of the workflow—every field, every disclosure, every branch, every termination criterion—is enumerated in one static system message at agent-construction time, and the model is then expected to keep the whole thing in mind, decide what step is next, and emit the right tool call at the right moment. The second is to maintain *several* agent configurations, each with a different system prompt, and treat per-turn control as a routing problem: each turn picks one of the configurations to generate the response. Both architectures treat the system prompt as the primary unit of control. They differ only in whether there is one large prompt or several.

We think this framing concedes too much. There is a third axis of control that neither architecture exploits: the *conversation history itself*—specifically, the messages that the runtime adds to the LLM's context just before each request. If we can engineer those messages dynamically, we can move the workflow's procedural detail out of any one upfront prompt and into a stream of *small, just-in-time guidance messages* delivered at the moment they are relevant. The agent retains a single conversational role (avoiding the multi-configuration routing problem and the term-of-art ambiguities that come with it), but its context for each request is no longer fixed ahead of time. The natural mechanism for authoring those just-in-time messages is the tool layer: a tool call's return is already a message that lands in the LLM's input on the next request, and the Model Context Protocol's *elicitation*

pattern already proposes a typed shape for it (a user-facing prompt plus a JSON-Schema fragment). We therefore use MCP-style elicitations as our model for tool-authored, just-in-time guidance.

This shifts the design problem. Instead of asking "which system prompt do I select for this turn?", we ask "which tool, with what arguments, should be called now to author the next guidance message?"—and, recursively, "what mechanism (a checklist, a state machine, a script with a model in the loop, a higher-level orchestrator) produces those messages reliably?" The same workflow can be guided by very different mechanisms behind the tool, with very different reliability and flexibility properties. We name several of them in Section 4 and study two pairings on the same benchmark.

What we argue.

- For workflows with hard procedural constraints, an architecture in which a single conversational role is steered by tool-authored, just-in-time guidance messages is a competitive alternative to both the long-upfront-prompt architecture and the multi-system-prompt-selection architecture.
- This architecture leaves the LLM in the conversational loop while moving the workflow's checklist, ordering rules, and branch decisions into the tool layer, where they can be made deterministic and inspected by program logic.
- How the guidance messages are produced behind the tool—rigidly versus loosely, programmatically versus model-mediated—is itself a design space with its own reliability and flexibility trade-offs, distinct from the question of *whether* guidance arrives upfront or just-in-time.

What we measure. We evaluate two slices of this design space empirically on a constrained support-intake benchmark.

Experiment one (Section 8) isolates the *form-filling* layer. It compares a long-upfront-prompt baseline against a guided arm in which structured capture is mediated by a single inline tool, `collect_field(name, value?)`, whose return is an MCP-style elicitation payload (first call) or an `accepted/elicit/blocked` record (second call). Both arms share task, validators, simulator, and model. Reported at $N = 100$ per persona per arm in two simulator client modes.

Experiment two (Section 9) isolates the *orchestration* layer. Both arms have the same form-filling layer, implemented as nested sub-tools that the agent invokes from inside an outer orchestrator tool. The orchestrators differ in how rigidly they enforce the conversation's phase ordering: a *rigid* orchestrator insists on a fixed phase sequence; a *loose* orchestrator allows the agent to defer compliance-permitting phases when the user is steering elsewhere. We add a third arm using the same upfront-instructions baseline

as experiment one, and run all three under an *impatient* simulator client mode that hangs up if its preferred topic is repeatedly deflected.

Scope. This is a single-task, single-model-family, simulated-user study. We do not claim the percentage-point magnitudes generalize. We claim that the qualitative directions are consistent with the architectural argument: where procedural detail lives, and at what granularity it reaches the LLM, is itself a reliability lever, and tool-authored just-in-time guidance is a viable place to put it.

Research questions.

RQ1 (form-filling)
Under matched task, validators, and personas, does shifting the form-filling layer from a long upfront checklist to per-step tool-authored elicitations improve strict task success and completion?

RQ2 (orchestration)
Holding the form-filling layer constant, how does a rigid orchestration policy compare against a looser one when the simulated user has a strong topic preference and a finite patience budget?

RQ3 (mechanism)
What classes of failure does each layer's intervention remove mechanistically, and what failures remain that no in-context guidance can address?

2 Background and motivation

2.1 What "guided conversations" means in production

A useful way to read the rest of this paper is to fix a working definition of *guided conversation*. We mean: a multi-turn LLM-driven conversation that must satisfy an external definition of *done*, follow a prescribed (sometimes branching) sequence of phases, and record specified pieces of structured data along the way. Commerce-side customer support intake, sales qualification, regulated triage, claims initiation, structured onboarding, and compliance interviews are typical examples. They share three properties that ordinary chat does not:

- the dialogue exists to drive a downstream system (a ticketing system, a fulfillment workflow, a regulatory filing) that consumes structured fields, not transcripts;
- the workflow has hard ordering and branching rules—some questions only matter if an earlier question went a particular way, some disclosures must be made before any account identifier is read aloud, and so on; and

- the workflow has a strict definition of done—"we collected every required field, in the right order, in a usable format, and the user gave the required acknowledgments"—that the conversation either meets or doesn't.

Building such a thing on top of a chat-completions runtime is now common practice, but the question of *how to control the conversation* is still unsettled. The two architectures named in the introduction—one long upfront system prompt, or several upfront prompts plus per-turn routing—exhaust most of the field's current vocabulary for the problem.

2.2 The two prevailing architectures and what they assume

The single-upfront-prompt architecture. The dominant pattern in early agentic work and in many off-the-shelf agent frameworks is to write the entire workflow into one static system message. The system prompt enumerates the steps, the disclosures, the branch rules, the tool surface, and the completion criterion; the LLM is then expected to internalize all of it and decide, on each turn, what to do next. This pattern's appeal is its simplicity: there is one place to read or change the workflow, and the agent runtime is a thin wrapper around chat completions. Its weakness, learned the hard way in commerce-side deployments where guided-conversation reliability is itself the product, is that it asks one stochastic LLM call to be both a conversational policy and the workflow's own checklist; under realistic workloads, completed conversations may still skip required disclosures, omit branch-specific fields, or burn the turn budget cycling between recovery attempts.

The multi-system-prompt-selection architecture. A second pattern, often described in the literature with terms like "multi-agent systems" or "agent routing", maintains several agent configurations, each with its own system prompt (and sometimes its own tool surface), and routes per-turn control to one of them. The control structure is typically expressed as a state machine, a planner, or a learned router that selects which configuration to use for the next response. We deliberately avoid the term *multi-agent* for this pattern in the rest of the paper, both because the configurations being switched between are typically not independent agents in any operational sense and because the term invites a definitional debate that is orthogonal to our claims; we will say *multi-system-prompt selection* instead. This pattern's appeal is that it lets each configuration stay short and focused; its corresponding cost in commerce-side deployments is that the design problem now factors into a router specification *and* K system-prompt specifications, and the seam between them is itself a place where reliability degrades. Its central assumption is that per-turn control reduces to selecting one of K static system prompts.

What both assumptions miss. Both patterns treat the system prompt as the only seat of context engineering. They differ on whether there is one chair or several. Neither exploits the fact that the conversation history—the sequence of user-role and tool-role messages that precede each LLM request—is also under the runtime's control and can be engineered dynamically. If we author the right messages there at the right moment, we do not have to fit the workflow into either an upfront blob or a finite menu of upfront blobs. Figure 1 sets the three architectures side-by-side at the level of one LLM request's input context.

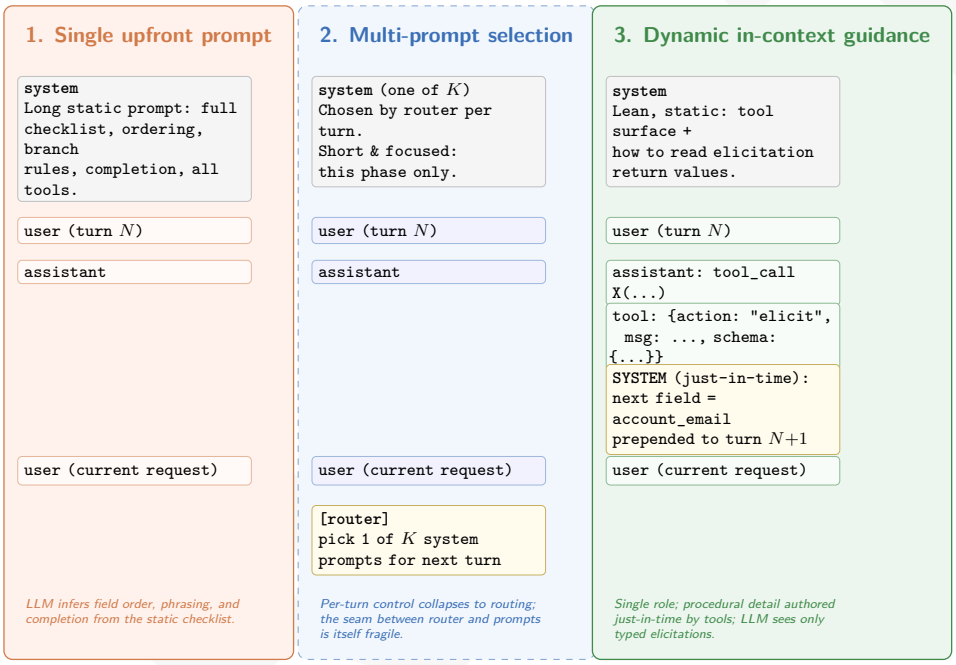


Figure 1: The three control architectures contrasted in this paper, each shown as the input context one mid-conversation LLM request would see under that architecture. (1) Single upfront prompt: one large static `system` message carries the full workflow; the LLM is asked to be both the conversational policy and the workflow's own checklist on every turn. (2) Multi-prompt selection: each turn picks one of K short static prompts; per-turn control reduces to a routing decision. (3) Dynamic in-context guidance: a single conversational role with a lean static prompt; procedural detail is authored just-in-time by tools (small typed elicitation payloads in chat history) and by short `SYSTEM:` hints prepended to user-role messages. The two experiments in this paper evaluate slices of (3) against (1); a controlled comparison against (2) is the natural follow-up (Section 11).

2.3 An alternative: dynamic, in-context, tool-authored guidance

The alternative we explore in this paper keeps the LLM in a single conversational role with one (lean) static system prompt and shifts the procedural-control workload into the conversation history. The runtime authors short, structured guidance messages *just-in-time*—when they are needed, with the content the workflow needs delivered at that moment—and inserts them into the LLM's input for the next request. The natural authorship surface for those messages is the tool layer: a tool's return is already a message that the next LLM request will see. The MCP elicitation pattern (a tool return shaped as "user-facing message + JSON-Schema fragment + a short instruction for the model") gives us a typed contract for what those just-in-time messages look like.

The change of frame is significant. The system prompt is no longer the workflow's checklist, the workflow's branch rules, or the workflow's termination criterion; it is the protocol document for how the agent participates in the workflow. The procedural detail itself lives in the procedure that authors the elicitations and in the validator code behind the tool.

This relocation of procedural detail has three properties that are worth naming early:

- It is *deterministic where it can be*: ordering, branching, and validation are program logic in the procedure that authors the messages, not constraints the LLM is asked to remember.
- It is *small at every step*: each guidance message is short, narrowly scoped, and disposable; the LLM is not asked to integrate a whole workflow on every turn.
- It admits multiple *back-end mechanisms*: behind the tool's elicitation messages can sit a static checklist, a state machine, a script, an LLM-condensed planner, or a higher-level orchestrator that delegates to nested tools. The architecture is the choice of where guidance comes from; the mechanism behind the tool is a separate design choice.

The two experiments in this paper measure two slices of this design space. Experiment one (Section 8) compares the long-upfront-prompt baseline against a guided arm whose form-filling layer is mediated by a single tool returning MCP-style elicitations; experiment two (Section 9) holds form-filling constant and varies the rigidity of the *orchestrator* that authors phase-level guidance.

3 Two ways to deliver form-filling guidance

This section makes the architectural distinction concrete on the form-filling layer. It is one specific instance of the broader contrast set up in Section 2:

the workflow's per-field obligations can either be enumerated in the LLM's upfront prompt, or authored just-in-time by a tool. Both control policies that we compare in experiment one exist to satisfy the same workflow obligations (recording disclosures, asking for an email, confirming a billing acknowledgment, and so on). They differ only in *how much of the procedural detail is in the LLM's upfront prompt versus delivered to the LLM, in-context, by a tool at the moment the procedure has a next thing to do.*

Upfront instructions (unguided). In the unguided policy, the orchestrating procedure ships the LLM a long static system prompt that enumerates every step of the workflow—which disclosure to read first, which fields to record, which fields are branch-specific, in what order—and exposes a generic tool surface that includes `record_field`. On every turn the model is responsible for keeping the whole checklist in mind, deciding which step is next, phrasing the user-facing message, and emitting the right tool call. The procedure runs validators inside each tool handler but does not direct the conversation. This is the most direct expression of the now-common "upfront-prompt" agentic style.

In-the-moment, tool-mediated elicitation (guided). In the guided policy, the LLM stays in the user-facing conversational loop, but each act of structured capture is mediated by an inline tool, `collect_field(name, value?)`, whose return shape is an MCP-style elicitation payload. On the first call (no `value`), the tool returns `{action="elicit", message, requested_schema, instruction}`: a user-facing prompt the LLM is instructed to relay verbatim, plus a JSON-Schema fragment describing the value the procedure expects, plus a short note telling the model that its next tool call must submit the user's reply. On the second call (with `value`), the tool validates the reply against the schema and returns either `{action="accepted", recorded, value, next_hint}`, `{action="elicit", error, message, requested_schema}` (a re-ask whose `message` now embeds the validation error so the LLM can re-prompt the user), or `{action="blocked", reason, do_first}` (when the call would violate ordering or branch constraints). On every turn, the procedure also prepends a one-line `SYSTEM:` hint to the user-role message naming the field that is still required next. The upfront system prompt is correspondingly lean: it describes the four available tools and tells the model that ordering and branch rules will be returned by the tools at the moment they apply.

The conceptual move is to push procedural detail out of the upfront prompt and into a tool that hands the model a small, schema-typed elicitation each time the procedure has a next step. The LLM is still the conversational policy; it is no longer also asked to be its own checklist.

Hooks and elicitations are orthogonal. The guided architecture above bundles two concepts that are in fact independent. A *hook* is a position in the runtime where code runs — an extension point that fires at a specific event (pre-X, post-X, on-X) and may observe, modify, or block the event. A hook answers the question *where and when* code intervenes. An *elicitation*, in the sense inherited from the MCP spec, is a typed payload shape carrying a user-facing `message`, a `requestedSchema`, an `action` discriminant, and implementation-specific `instruction` text. An elicitation answers the question *what shape* a message takes as it flows through the runtime. Hooks are plumbing; elicitations are cargo. The guided arm above installs two distinct hooks: a pre-user-message hook prepends the one-line `SYSTEM:` block, and a tool-return hook emits the MCP-style elicitation payload from `collect_field`. The first payload is free text; the second is a typed payload with a schema. One hook, one elicitation — the fact that the two are not the same shape is the proof that hooks and elicitations are independent concepts. Hooks exist without elicitations trivially (git `pre-commit`, web-framework middleware — all hooks; none MCP-shaped); elicitations, however, require some hook-like carrier to fire and deliver them. Elicitations presuppose hooks; hooks don't presuppose elicitations.

Two figures make the difference concrete. Figure 2 shows the per-step interaction trace for one representative capture (asking for the customer's email) in each arm. Figure 3 shows what each arm's LLM actually sees in its input context per request—in particular, that the guided arm's LLM no longer carries the upfront checklist but does see the elicitation payloads it received from `collect_field` as part of its growing chat history. Figure 4 then summarizes the architecture at a higher level.

3.1 Per-step timing

The first comparison is at the level of one structured capture, abstracted from the surrounding workflow. Figure 2 traces the messages exchanged between the procedure, the LLM, and the simulated user as each policy attempts to record a single field (the customer's email). The unguided panel shows the upfront-prompt style: the LLM is responsible for deciding when to ask, phrasing the question from its checklist, interpreting the free-text reply, and emitting the `record_field` tool call. The guided panel shows the in-the-moment style: the LLM calls `collect_field` with just the field name and gets back an elicitation payload; it relays the payload's user-facing `message` verbatim; on the user's reply it calls `collect_field` again with the value and the procedure validates and stores it. The LLM is in the loop in both arms; what differs is whether the *guidance* for that step came from the upfront prompt or from a tool return.

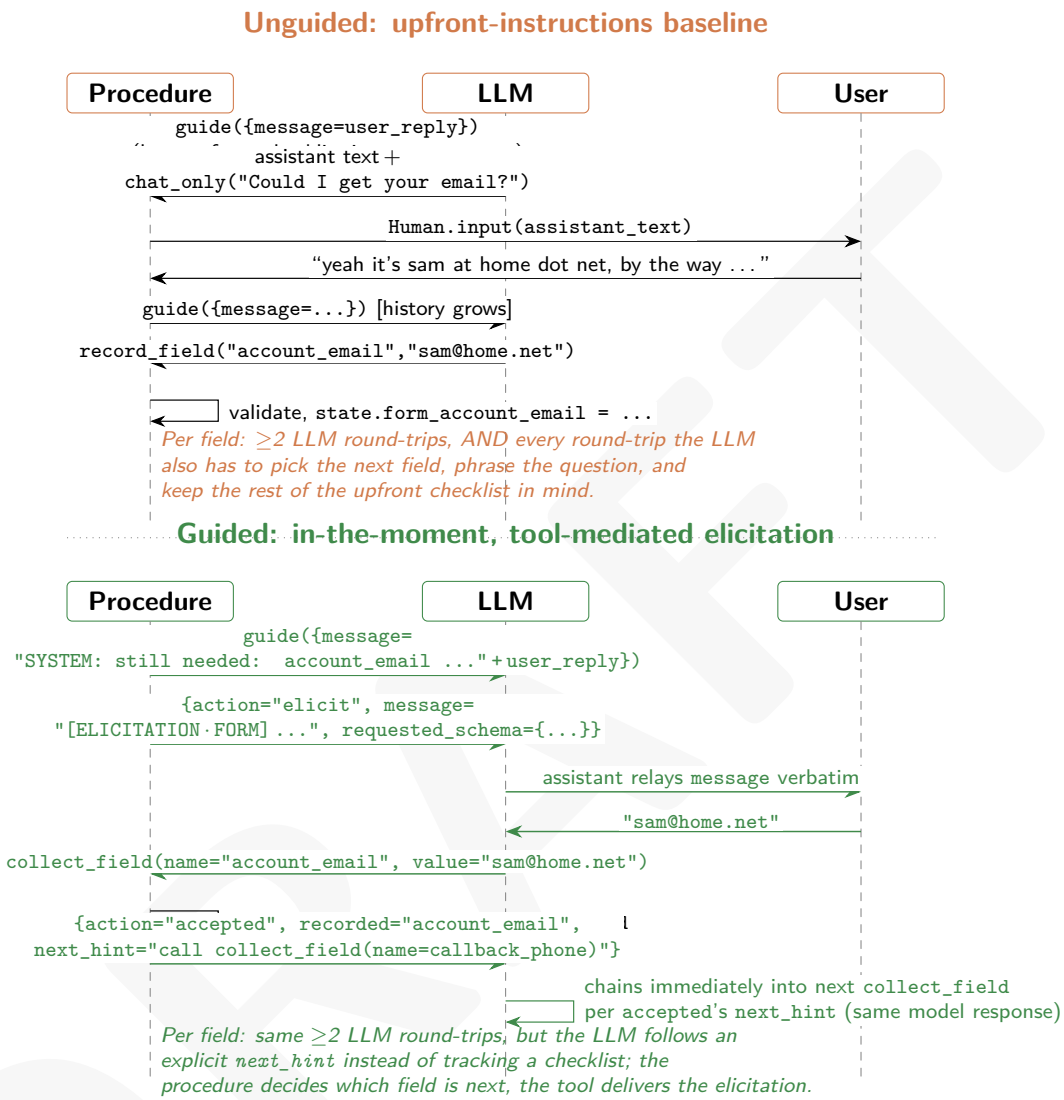


Figure 2: Per-step interaction for capturing one structured field (`account_email`). Top: in the upfront-instructions baseline the LLM is the workflow's checklist *and* its conversational policy; it must decide which field is next from the long static prompt, phrase the question, interpret the free-text reply, and emit a generic `record_field` call. Bottom: in the in-the-moment, tool-mediated arm the LLM stays in the conversational loop but the procedural guidance for each step is delivered by the `collect_field` tool—a typed elicitation payload (user-facing prompt + JSON-Schema fragment) on the first call, an `accepted`/`elicit`/`blocked` record on the second. The LLM relays the elicitation prompt verbatim and then submits the user's reply through the same tool. Both arms run roughly the same number of LLM round-trips per field; the difference is what the LLM has to do on each round-trip—track the upfront checklist itself in the unguided arm, or follow the procedure-authored `next_hint` in the guided arm. On `accepted`, the guided arm chains immediately into the next `collect_field` call inside the same model response; that chained call is the dotted self-loop in the lower panel.

3.2 What the LLM actually sees per request

The second comparison is at the level of one LLM request, mid-conversation. Figure 3 shows the input context—static system prompt plus accumulated chat history—that each arm sends. Three implementation details, all faithful to the code, appear here for the first time and matter for how the rest of the paper should be read. First, the agent runtime sets a single static `system`-role message at agent-construction time; both arms use this slot, but the unguided arm's prompt is much longer because the entire workflow checklist lives there, while the guided arm's prompt only describes the four available tools and a short usage protocol. Second, the guided arm communicates the next-required-field hint by appending a one-line `SYSTEM:` block to each user-role turn (amber highlight); ordering and branch rules that the unguided arm hard-codes into its checklist are returned by `collect_field` on demand instead. Third, the guided arm's chat history contains the actual elicitation payloads the LLM received from the tool—the same `message` text it relayed to the user, plus the validated `accepted` records that came back when it submitted user values. The values *are* in the LLM's context, but they got there through tool calls the LLM made itself, not through a procedure-injected echo.

3.3 Architecture summary

Figure 4 steps back to the architecture level. Both arms share the same task substrate—validators, simulator, and model—and differ only in how procedural detail reaches the LLM (one upfront blob versus per-step typed elicitations from `collect_field`). This is the form-filling-layer instance of the broader argument made in Section 2; experiment one (Section 8) is the empirical comparison.

4 Engineering the guidance messages

If the architectural choice is "where does procedural detail live—upfront prompt(s), or just-in-time tool returns?", then choosing the just-in-time arm forces a second design choice: *how* are the just-in-time messages produced? The tool is the authorship surface; the mechanism behind the tool can take many forms. This section names five and points to where in this paper or its supplementary code each one appears.

We treat each vignette as a sketch, not a benchmark in itself. The point is that the same outer architecture (single conversational role, tool-authored elicitations) is compatible with very different back-ends, and that those back-ends sit on a real spectrum from rigid to loose, from purely programmatic to model-mediated.

Unguided: long static `system` + full chat history

system (persistent)

```
BASE_SYSTEM_PROMPT
You are a careful customer support agent
for a SaaS
product.
CHECKLIST (follow in order; you may
greet first):
1) Recording/privacy: Tell the user
calls may be recorded
  ... then call
record_compliance(recording_privacy).
2) issue_category: Record exactly
general, billing, or
  technical ...
3) Core fields: account_email,
issue_summary, callback_phone.
4) Branch: If technical -- device_model.
If billing -- fee
  disclosure, then
billing_charge_acknowledged.
5) Explain a short resolution plan; then
plan_approval = yes.
6) Call done only when every required
field is satisfied.
TOOLS (exactly one per turn):
record_compliance |
  record_field | chat_only | done. ...
```

user

```
Hi, I'm calling about my account.
```

assistant

```
Thanks for calling. For quality and
training, this
call may be recorded ...
tool_call:
record_compliance(recording_privacy,
...)
```

user

```
OK. My email's sam at home dot net.
By the way I keep getting weird charges
on my card.
```

assistant

```
Got it.
tool_call:
record_field("account_email",
         "sam@home.net", ...)
```

user (current request)

```
And it's three of them, all last month
...
```

Guided: lean static `system` + tool-mediated elicitation in chat history

system (persistent)

```
GUIDE_SYSTEM_PROMPT
You are a careful customer support
agent.
Tools (exactly one per turn):
  collect_field(name [, value])
  record_compliance | chat_only | done.
For collect_field: first call returns an
elicit
payload; relay its 'message' verbatim.
Second call
with the user's value returns
accepted/elicit/blocked.
```

user

```
SYSTEM: Still needed: account_email,
callback_phone, fee_terms disclosure,
...
Next field to elicit: account_email.
OK. Could I get my email?
```

assistant

```
tool_call:
collect_field(name="account_email")
```

tool (collect_field → elicit)

```
{action: "elicit",
  message: "[ELICITATION·FORM] Account
email
    Please provide ...(Required:
account_email)",
  requested_schema: {type:"string",
format:"email"}}
```

assistant

```
[ELICITATION · FORM] Account email
Please provide the email on the account.
(Required: account_email)
```

user

```
marcus@work.io
```

assistant

```
tool_call:
collect_field(name="account_email",
  value="marcus@work.io")
```

tool (collect_field → accepted)

```
{action: "accepted",
  recorded: "account_email",
  value: "marcus@work.io",
  next_hint: "Next: callback_phone."}
```

Figure 3: The LLM's input context for one mid-conversation request, under each arm. Both arms send a single static `system` message at the top of every request. What differs is (a) the size of the static prompt and (b) the shape of the rest of the context. The unguided arm's system prompt carries the entire workflow checklist; the guided arm's prompt only describes the tools and how they signal next steps. The guided arm's chat history then contains the actual `collect_field` tool calls the model emitted, the elicitation payloads the procedure returned (green-bordered cards), the user's typed reply, and the `accepted` record returned on submission. The procedure also prepends a one-line `SYSTEM:` hint (amber) to each user-role turn naming the next required field, so the model never has to infer ordering from a checklist held only in its prompt.

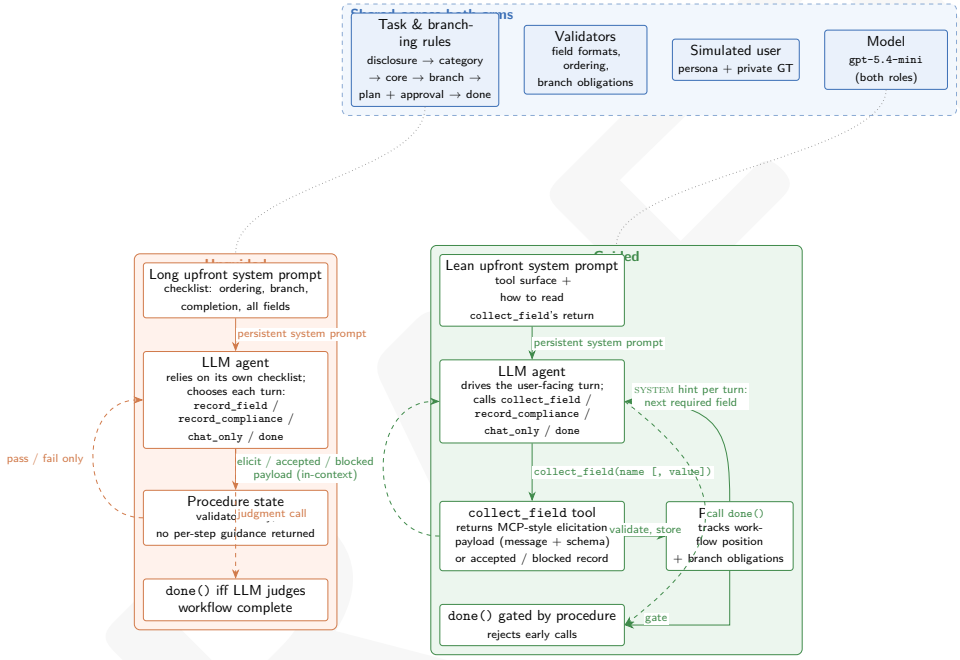


Figure 4: The same diagram at the architecture level. Both arms share task, validators, simulator, and model. The unguided arm folds the workflow checklist into the upfront system prompt and exposes a generic `record_field` tool; the guided arm trims the upfront prompt and exposes a single `collect_field(name, value?)` tool that returns MCP-style elicitation payloads at the moment the procedure has the next step. The LLM stays in the user-facing conversational loop in both arms.

4.1 Vignette 1: a compiled-in static checklist

The simplest authorship mechanism is a static, hand-coded checklist embedded in the tool's procedure: a fixed list of fields with their order, validators, and per-field elicitation messages. The tool's job each turn is to look at procedure state, pick the next required field, and return its elicitation payload. No model is involved in producing the guidance message.

This is the back-end used by experiment one's guided arm (Section 6). Its strength is that the workflow is fully deterministic where the workflow itself is fully determined: ordering is enforced by program logic, validation is enforced by per-field validators, and the LLM never has to integrate the whole checklist on a single turn. Its weakness is the dual: the back-end inherits the inflexibilities of any compiled-in checklist, including a poor ability to handle phase-level conversational dynamics (out-of-order user requests, deferrals, topic preferences).

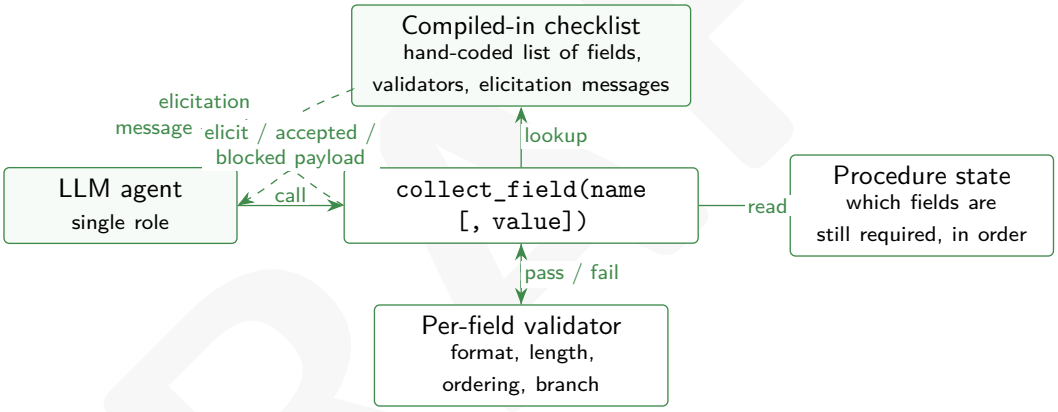


Figure 5: Vignette 1: a compiled-in static checklist behind the elicitation tool. Each tool call inspects state, picks the next required field, and returns its elicitation payload. The author of the message is the procedure code itself.

4.2 Vignette 2: a state machine

A more general mechanism is to express the workflow as a state machine: nodes are workflow phases, edges are guarded transitions that depend on procedure state and user inputs. The tool authors the next elicitation by reading the current node, checking which outgoing edges are eligible, and emitting the elicitation message attached to the chosen next node.

This back-end suits workflows whose "shape" is known but whose order can be context-sensitive. The state-machine description can be authored separately from the tool wiring, which is useful when domain experts (rather

than agent engineers) own the workflow's shape. It still enforces ordering and branching deterministically, with the same strengths as the compiled-in checklist, plus a clearer separation between "what the workflow looks like" and "how the agent invokes it".

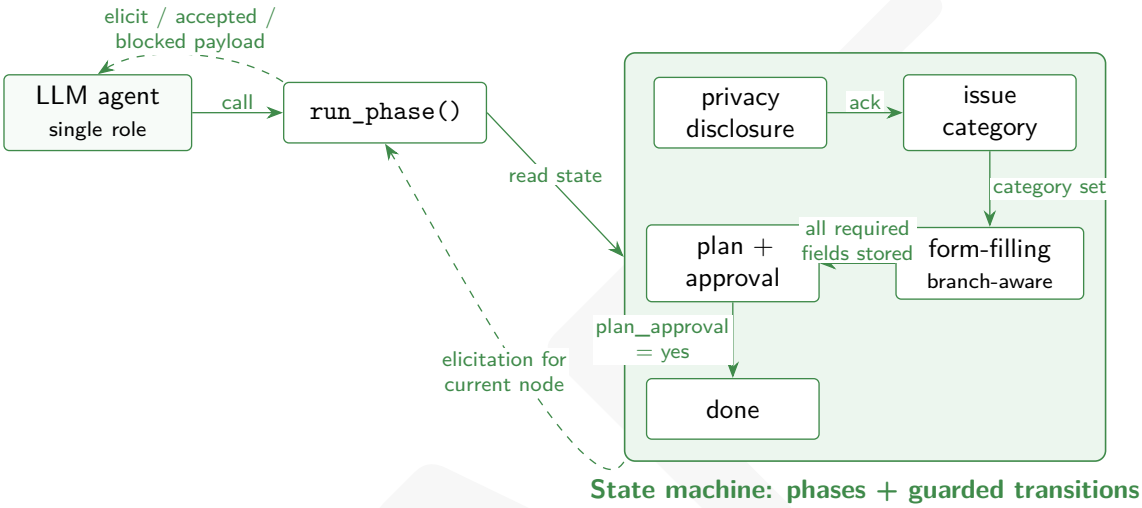


Figure 6: Vignette 2: a state machine behind the elicitation tool. The current node names the next phase; outgoing edges encode ordering and branching constraints; the tool returns the elicitation attached to the chosen next node.

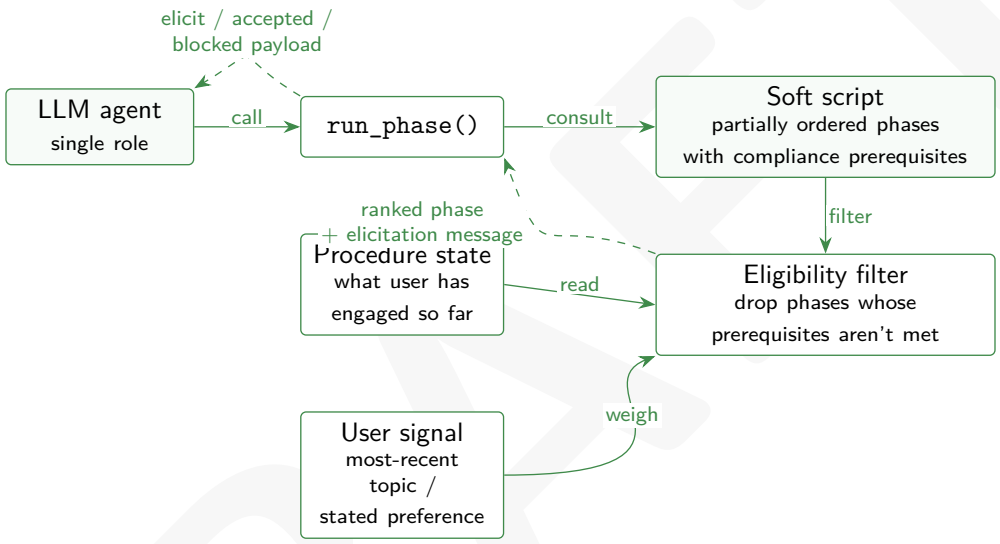
4.3 Vignette 3: a soft script

A looser back-end is a soft *script*: a partially ordered list of phases or talking points, plus a small piece of procedure logic that decides which one to surface next based on what the user has done. This back-end gives up some determinism (the order can vary) in exchange for accommodating real-world conversational dynamics: users who jump ahead, ask for things in a different order, or want to discuss a particular topic before completing required work.

The looseness is bounded. The script names which phases are blocked by which (e.g. "no account-detail phases until the privacy disclosure has been delivered"), so the procedure can refuse phases that would violate compliance prerequisites even when accommodating the user's preferences. Experiment two's loose orchestrator (Section 9) is an instance of this back-end, scaled up to whole conversation phases rather than individual fields.

4.4 Vignette 4: an LLM-condensed planner

So far the back-end has been program logic. There is no rule that says it has to be. A fourth option is to put a small, focused LLM call *behind* the



Same outer contract as vignettes 1–2; the back-end picks which eligible phase to surface based on user signal, not on a fixed transition graph.

Figure 7: Vignette 3: a soft script behind the elicitation tool. A small piece of procedure logic ranks the eligible phases against what the user has just done and surfaces the ranked phase's elicitation. Compliance prerequisites filter out phases the procedure refuses to author.

tool: given the current procedure state and the recent user message, generate the next guidance message. The outer agent still sees only the elicitation payload; the inner LLM is invisible to it.

The strength of this back-end is that it can produce richer, more context-sensitive guidance than a pre-written elicitation. The cost is that we have re-introduced model stochasticity at exactly the layer we tried to make deterministic. An interesting middle ground is to use the inner LLM only for natural-language phrasing of guidance whose content is decided programmatically ("the next phase is X; phrase the elicitation as a single short question in plain English"), keeping the structured contract intact.

We do not study this back-end in either experiment, but it lives on the same architectural axis: the message is authored by the tool, just-in-time, and reaches the LLM as a typed elicitation in the next request's context.

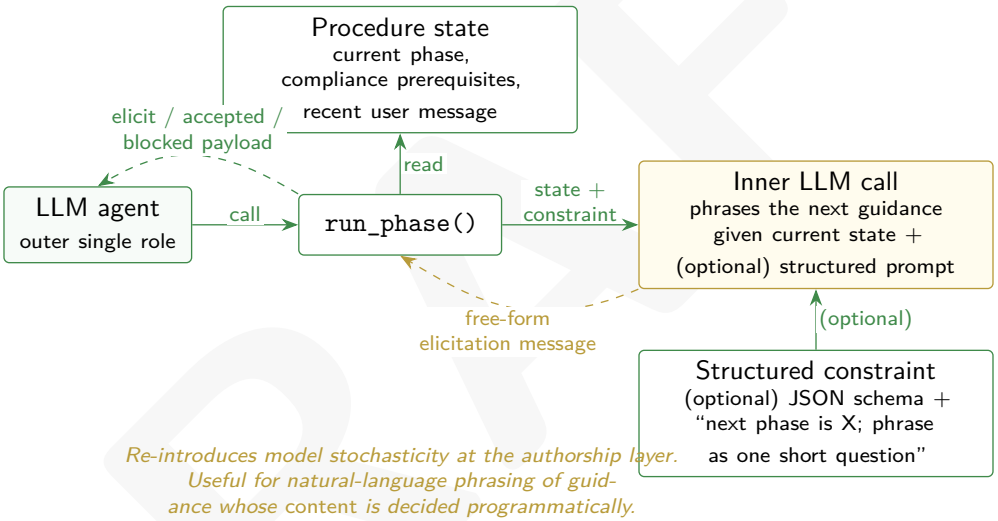


Figure 8: Vignette 4: an LLM-condensed planner behind the elicitation tool. An inner LLM call authors the next guidance message from current state and (optionally) a structured constraint; the outer agent still sees only the typed elicitation. The amber edge marks the layer where model stochasticity is re-introduced.

4.5 Vignette 5: an orchestrator with continuations

The most expressive back-end is to allow the tool's elicitation to itself name a *sub-tool* the agent should call before returning to the orchestrator. The agent acts as a scheduler: it calls the orchestrator, the orchestrator returns "call sub-tool T with these arguments and then call me again with the result", the agent calls T, T returns its own elicitation, the agent answers T, T returns

a result to the agent, and the agent re-calls the orchestrator with the result. The conversation history is the call stack; the `next_hint` or `resume_token` field in the orchestrator's elicitation payloads is the return address. We call this style *continuation-passing*, by analogy.

Continuation-passing is what makes *nested* tool-mediated guidance possible. Experiment two's orchestrator arms use this style: the outer orchestrator authors the conversation's phase guidance, and at appropriate points hands the agent off to nested sub-tools (e.g. a form-filling sub-tool that itself uses the compiled-in-checklist back-end). The back-ends from vignettes 1–4 can therefore be *composed*: a state-machine orchestrator at the top can delegate field capture to a compiled-in-checklist sub-tool, which in turn might delegate phrasing to an LLM-condensed sub-tool. The architectural commitment to single-conversational-role is compatible with multi-layer authorship.

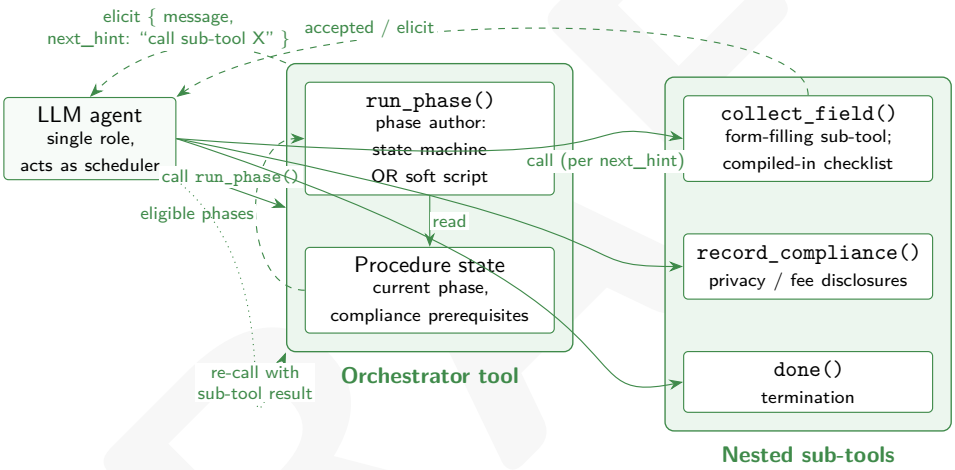


Figure 9: Vignette 5: an orchestrator tool whose elicitation payloads can point at sub-tools the agent should call before returning. The agent acts as a scheduler; the conversation history is the call stack; the orchestrator's `next_hint`/`resume_token` field is the return address. Experiment two's two orchestrators (Section 9) use this style.

4.6 Where on this spectrum the experiments sit

The five vignettes are points on a continuum from rigid-and-deterministic to loose-and-model-mediated. The two experiments in this paper sample the spectrum:

- Experiment one's guided arm uses vignette 1 (compiled-in checklist) at the form-filling layer, with no orchestrator above it; the entire conversation is a single field-filling loop.

- Experiment two's two orchestrator arms both use vignette 5 (orchestrator with continuations) at the top, delegating to vignette 1 (compiled-in checklist) for form-filling. The two arms differ in whether the orchestrator's own logic is rigid (vignette 2-style state-machine ordering) or loose (vignette 3-style soft script).
- Experiment two also includes a long-upfront-prompt baseline arm, for the same reason experiment one does: it anchors the comparison to the field's current default.

Other points on the spectrum—purely state-machine workflows, LLM-condensed planners, hybrids of all of the above—are open design questions we return to in Section 11.

5 Benchmark

5.1 Task and constraints

The benchmark models a phone-style customer support intake. A run is a multi-turn conversation between the agent and a simulated user. Successful intake requires, in order or with documented enforcement:

1. recording/privacy disclosure (compliance) before any account identifier;
2. issue category classification, restricted to `general`, `billing`, or `technical`;
3. core fields: `account_email`, `issue_summary`, `callback_phone`;
4. branch-specific fields (`device_model` for technical; fee disclosure plus `billing_charge_acknowledged` for billing);
5. a brief plan explanation followed by an explicit `plan_approval` acknowledgment;
6. a final `done` call.

These rules are enforced by Lua validators and procedure-side guards. Field-level constraints include format checks (phone `XXX-XXX-XXXX`, plausible email), minimum lengths, and exact-string acknowledgments (`yes`). The same validator set is used in both arms.

5.2 Personas

Three personas isolate different stress patterns (Table 1). Each persona has a private ground-truth dictionary covering all branch-specific requirements; ground truth is held constant across runs.

Table 1: Personas and the structured fields each one ultimately needs to disclose.

Persona	Branch	Distinctive challenge
`support_rambler`	general	buries answers in long stories
`support_billing`	billing	extra fee disclosure + ack + approval
`support_technical`	technical	extra device-model branch field

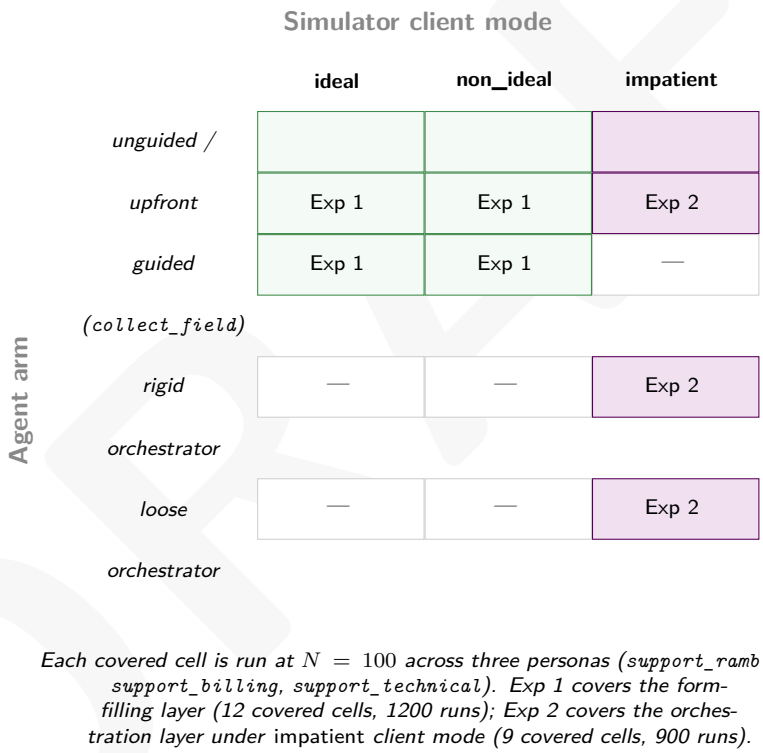


Figure 10: Experimental design as an arm × simulator-mode coverage matrix. Each green cell is part of Experiment 1's form-filling-layer comparison (Section 8); each violet cell is part of Experiment 2's orchestration-layer comparison (Section 9). Empty cells are not run; the rationale for each gap is given in the cited sections. Every covered cell is run at $N = 100$ across the three personas in Table 1.

6 Control policies, concretely

Section 3 introduced the conceptual contrast and Figures 2–4 illustrate the mechanism. This section pins down the implementation surface so the comparison is reproducible.

Unguided. The unguided arm exposes the tool surface `record_field`, `record_compliance`, `chat_only`, `done` to the LLM, with `tool_choice = required` so every turn must emit exactly one tool call. The system prompt is a long upfront checklist enumerating every disclosure, every field, the branch rules, and the completion criterion. The procedure runs validators inside each tool handler but does not initiate capture; it advances state in response to whatever the model emits.

Guided. The guided arm exposes the tool surface `collect_field`, `record_compliance`, `chat_only`, `done`, also with `tool_choice = required`. `record_field` is intentionally absent; structured capture is structurally only available through `collect_field`. The system prompt is correspondingly lean: it describes the four tools and tells the model how to interpret `collect_field`'s three return shapes (`elicit`, `accepted`, `blocked`). On every turn the procedure prepends a one-line `SYSTEM:` hint to the user-role message naming the next required field; the same prepended block also names any pending compliance disclosures. When the model calls `collect_field(name="`*X*`")` with no `value`, the procedure returns `{action="elicit", message, requested_schema, instruction}` and the model is expected to relay `message` verbatim to the user. When the model calls `collect_field(name="`*X*`", value="`*v*`")`, the procedure normalizes `v` (lightweight regex extraction for emails, phones, and category words), runs the field validator, and either stores it (returning `accepted` with a `next_hint`), re-elicits (returning `elicit` with the validation error embedded in the new `message`), or blocks the call (returning `blocked` with a one-line `do_first`). All ordering and branch enforcement is in the procedure; the LLM observes those rules as tool-return semantics rather than as upfront instructions.

What is held constant. Both arms use the same persona-grounded simulator, the same model (`gpt-5.4-mini`) for both the agent and the simulator, the same maximum-turn budget, the same set of field validators, and the same outcome classification. Only the agent-facing tool surface and the system prompt differ.

7 Simulated user and protocol

7.1 Simulator

We replace human input with `LLMHITLHandler`, a fast model role-playing the persona using the persona's private ground-truth dictionary. For ordinary conversational turns, the simulator generates user replies in character.

Behavior at elicitation prompts (*ideal* mode). When the LLM relays an elicitation prompt verbatim (recognized by the `[ELICITATION·FORM]` sentinel header used in Figure 2), the simulator's default *ideal* mode returns the requested ground-truth field value directly rather than improvising a paraphrase. This idealizes the user as a well-formed structured client that submits exactly the field requested, in the format the validator expects. The unguided arm receives the same treatment whenever the model itself produces user-facing text containing the same sentinel, so the simulator behavior is symmetric across arms. We discuss the implications of this idealization in Section 10.

Behavior at elicitation prompts (*non_ideal* mode). To probe the construct-validity gap that ideal-mode answers introduce, we also run a *non_ideal* simulator client mode (defined in `tests/llm_hitl_handler.py`). On the first ask of each elicitation field, the simulator samples one of four behaviors according to the module-level constants `P_CLEAN` = 0.35, `P_FORMAT_NOISE` = 0.30, `P_REFUSAL` = 0.15, and `P_WRONG_THEN_CORRECT` = 0.20. Format-noise responses inject realistic surface variants of the ground-truth value (e.g. a phone number whose first digit is incremented, or a device-model reply with only the trailing token); refusals return a short "I'd rather not" string; wrong-then-correct responses return a plausible-but-wrong value on the first ask and the clean ground-truth value on re-ask, exercising the procedure's retry loop end-to-end. The format-noise case in particular produces a class of well-formed-but-wrong values that no per-field validator can reject; it is what differentiates the two modes' headline numbers. We report results separately for the *ideal* and *non_ideal* client modes.

7.2 Outcome classification

Each rollout terminates with one of four outcomes (Figure 12):

- **infra_error**: execute/API failure;
- **incomplete**: ran without infra failure but never legitimately reached `done`;
- **completed_strict_fail**: reached `done` but at least one ground-truth field is wrong, missing, or malformed;

ideal

Cooperative structured client. At any [ELICITATION·FORM] prompt, returns the requested ground-truth field exactly, in the format the validator expects. No noise, no refusals, no rephrasing.

Knobs: (none)

Used in: Experiment 1

non_ideal

On the first ask of each elicitation field, samples one of four behaviors (subsequent asks converge to clean):

Knobs (per first ask):

- P_CLEAN = 0.35 — ground-truth, clean.
- P_FORMAT_NOISE = 0.30 — well-formed but wrong (e.g. phone with a bumped digit).
- P_REFUSAL = 0.15 — "I'd rather not."
- P_WRONG_THEN_CORRECT = 0.20 — wrong, then ground-truth on re-ask.

Used in: Experiment 1

impatient

Cooperative on values, opinionated on phase order. Each persona has a preferred_topic; the simulator tracks whether the agent engages with it or deflects.

Knobs:

- patience_budget $B = 3$ (initial).
- engagement with topic resets B.
- each detected deflection: $B - = 1$.
- when $B = 0$: simulator emits HUNG_UP_SENTINEL and the run terminates as hung_up.

Used in: Experiment 2

Figure 11: The three simulator client modes used across both experiments. All three share the underlying persona-grounded simulator and the same ground-truth dictionaries; they differ only in the behavioral knobs applied at [ELICITATION·FORM] prompts and (for *impatient*) in the patience budget tracked across the conversation. Numerical knob values are taken from `tests/llm_hitl_handler.py`; the *impatient* mode is what introduces the `hung_up` outcome class used in Section 9.

- **strict_ok**: reached `done` with every ground-truth field passing its per-field check (see below).

Per-field policy. Branch-critical and identifier fields (`account_email`, `callback_phone`, `issue_category`, `device_model`) require exact match against ground truth after lowercasing and trimming, and boolean acknowledgments (`billing_charge_acknowledged`, `plan_approval`) require an exact boolean match. `issue_summary` is treated as a free-text field and is required only to be non-empty with at least five characters; we do not impose semantic equivalence on it because no downstream system in the modeled workflow consumes the summary as a structured key.

7.3 Metrics

We report:

- **Strict success (excluding infra)**: `strict_ok` runs divided by non-infra runs;
- **Completion**: `strict_ok` + `completed_strict_fail`, divided by all runs;
- **Infra failure rate**: infra runs divided by all runs.

Experiment two adds one more rate, since the simulator's *impatient* client mode introduces a fourth terminal-outcome class (`hung_up`; Section 9):

- **Hung-up rate**: `hung_up` runs divided by all runs.

Confidence intervals are Wilson 95% intervals for each binomial proportion; deltas are reported in percentage points.

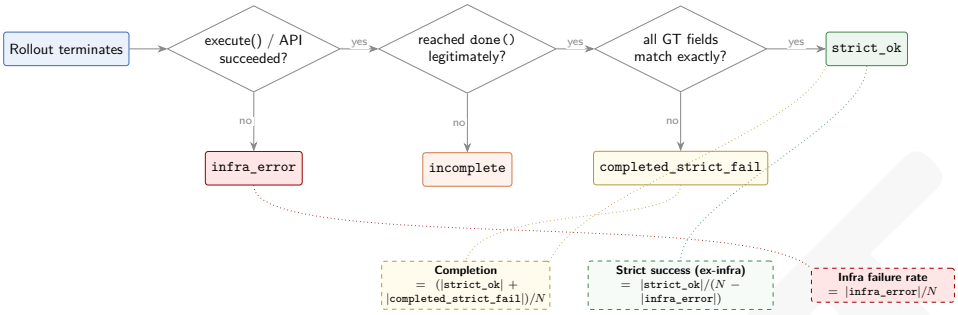


Figure 12: Outcome classification pipeline used in this study. Strict success counts only `strict_ok`; completion counts both `strict_ok` and `completed_strict_fail`; infra failure isolates execute/API errors so they cannot be charged to either policy. Experiment two adds a fifth class, `hung_up` (the simulator's patience budget reached zero before the agent called `done`); see Figure 11 for the simulator-mode definitions and Section 9 for the resulting outcome decomposition.

7.4 Study configuration

The canonical run reported below uses tag `llm_loop_full100_v1` with `RELIABILITY_RUNS=100` per persona per arm, `RELIABILITY_CONCURRENCY=10`, paired user simulation (`RELIABILITY_PAIR_USER_SIM=1`), and one-shot infra retry (`RELIABILITY_RETRY_INFRA=1`). Total executions: $2 \times 3 \times 100 \times 2 = 1200$ (two arms, three personas, $N = 100$ each, two simulator client modes).

8 Experiment one: upfront vs. tool-mediated elicitation

Experiment one isolates the form-filling layer. The two arms share task, validators, simulator harness, model, persona ground-truth, outcome classification, and turn budget; they differ only in how the procedural detail reaches the LLM, as introduced in Section 3. The unguided arm uses a long upfront checklist plus a generic `record_field` tool; the guided arm uses a lean upfront prompt plus the inline `collect_field(name, value?)` tool whose return is an MCP-style elicitation payload (see Figures 2–3).

8.1 Hypothesis

H1 (form-filling). Holding everything else equal, replacing the upfront checklist with per-step tool-authored elicitations will raise strict success and completion in every persona-by-mode cell, with the largest gains on personas whose workflows have the most ordering and branching constraints

(`support_billing`, then `support_technical`). The intervention should not raise infra failure, since both arms run the same agent runtime against the same model.

8.2 Headline outcomes

Table 2 reports strict success excluding infra failures; Table 3 reports completion. The full pilot is at $N = 100$ per persona per arm in two simulator client modes (*ideal*, *non_ideal*); zero infra failures occurred in any cell. The unguided baseline's strict-success rate spans 44–67% across cells; the guided arm reaches 97–100% under *ideal* and 54–88% under *non_ideal*. The largest *ideal*-mode gain is `support_billing` (+56pp), consistent with H1's prediction that the persona with the most ordering-sensitive obligations benefits most from offloading the workflow's checklist into the procedure. Under *non_ideal*, the gain narrows sharply on `support_rambler` (+5pp) and goes to roughly zero on `support_technical` (−1pp), while `support_billing` still gains 41pp. The mechanism behind the narrowing is the construct-validity slice we introduced *non_ideal* mode to expose: the format-noise behavior produces well-formed-but-wrong values (e.g. a phone number whose first digit is incremented, or a device-model reply with only the trailing token) that no per-field validator can reject. We discuss the implications later in this section.

Table 2: Experiment one: strict success (excluding infra), $N = 100$ per persona per arm; Wilson 95% intervals over the cell's N runs. Cells show `[lo, hi]` in percent; the rightmost column is the percentage-point gain of the guided arm over the unguided baseline.

Mode	Persona	Unguided	Guided	Δ (G−U)
ideal	`support_rambler`	[57.3, 75.4]%	[96.3, 100.0]%	+33.0pp
ideal	`support_billing`	[34.7, 53.8]%	[96.3, 100.0]%	+56.0pp
ideal	`support_technical`	[39.4, 58.7]%	[91.5, 99.0]%	+48.0pp
non_ideal	`support_rambler`	[58.3, 76.3]%	[63.6, 80.7]%	+5.0pp
non_ideal	`support_billing`	[37.5, 56.7]%	[80.2, 93.0]%	+41.0pp
non_ideal	`support_technical`	[45.2, 64.4]%	[44.3, 63.4]%	−1.0pp
Equal-weighted mean (*ideal*)		53.3%	99.0%	+45.7pp
Equal-weighted mean (*non_ideal*)		56.7%	71.7%	+15.0pp

8.3 Outcome decomposition

Outcome decomposition (Table 4) tells a sharper story than the rates alone. Two questions matter independently: (i) of the failed unguided runs, what

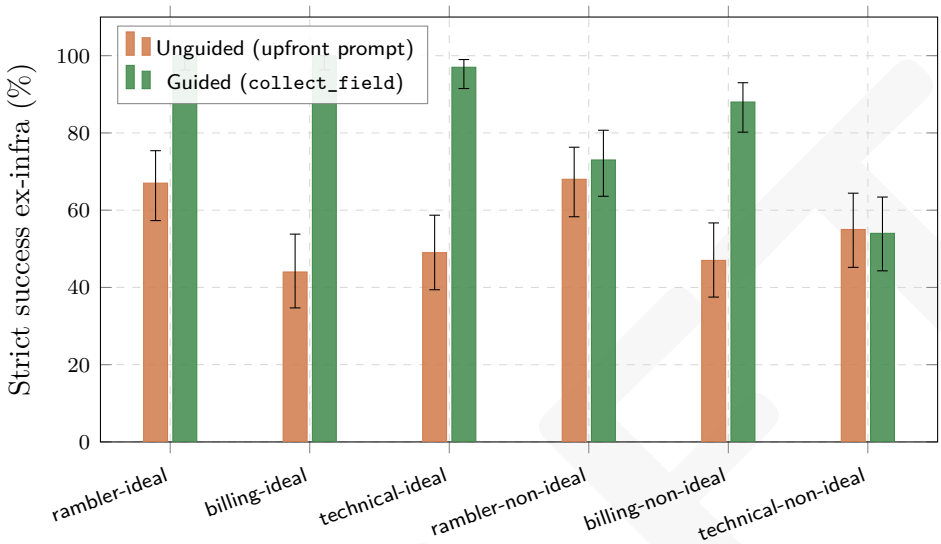


Figure 13: Experiment one: strict success ex-infra by (mode, persona) cell, with Wilson 95% intervals. Same numbers as Table 2; the visual makes the persona-by-mode structure of the gain immediately legible. The *ideal*-mode gains are uniformly large; the *non_ideal*-mode gains are concentrated on `support_billing`, with rambler near-tied and technical effectively unchanged.

Table 3: Experiment one: completion (procedure called `done`), $N = 100$ per persona per arm; Wilson 95% intervals.

Mode	Persona	Unguided	Guided	Δ (G−U)
ideal	`support_rambler`	[60.4, 78.1]%	[96.3, 100.0]%	+30.0pp
ideal	`support_billing`	[37.5, 56.7]%	[96.3, 100.0]%	+53.0pp
ideal	`support_technical`	[46.2, 65.3]%	[96.3, 100.0]%	+44.0pp
non_ideal	`support_rambler`	[63.6, 80.7]%	[96.3, 100.0]%	+27.0pp
non_ideal	`support_billing`	[43.3, 62.5]%	[96.3, 100.0]%	+47.0pp
non_ideal	`support_technical`	[49.2, 68.1]%	[96.3, 100.0]%	+41.0pp

fraction failed by getting stuck (`incomplete`) versus by submitting a wrong value (`completed_strict_fail`)?; and (ii) of the failed guided runs in *non_ideal* mode, what fraction reflect well-formed-but-wrong values that no validator can reject?

Table 4: Experiment one: outcome decomposition by arm. Counts of runs (out of $N = 100$ per cell) classified by terminal outcome: `strict_ok` (procedure called `done` and every required field strict-matched ground truth), `incomplete` (procedure ran out of turns without calling `done`), `completed_strict_fail` (procedure called `done` but at least one field did not strict-match), `infra` (runtime/API error).

Arm	Persona	strict_ok	incomplete	completed_strict_fail	infra
Unguided / *ideal*	support_rambler	67	30	3	0
Unguided / *ideal*	support_billing	44	53	3	0
Unguided / *ideal*	support_technical	49	44	7	0
Unguided / *non_ideal*	support_rambler	68	27	5	0
Unguided / *non_ideal*	support_billing	47	47	6	0
Unguided / *non_ideal*	support_technical	55	41	4	0
Guided / *ideal*	support_rambler	100	0	0	0
Guided / *ideal*	support_billing	100	0	0	0
Guided / *ideal*	support_technical	97	0	3	0
Guided / *non_ideal*	support_rambler	73	0	27	0
Guided / *non_ideal*	support_billing	88	0	12	0
Guided / *non_ideal*	support_technical	54	0	46	0

8.4 Failure fields and turns-to-resolution

Among unguided runs that finished but failed strictness (`completed_strict_fail`), we expect the dominant culprit to be `issue_category`, the choice point that determines every downstream branch-specific requirement; followed by `device_model` (the technical-branch field) and `billing_charge_acknowledged` (the billing-branch acknowledgment). Table 5 reports the measured distribution.

Table 5: Experiment one: mismatched fields tallied across all unguided `completed_strict_fail` runs (both client modes, all three personas; 28 such runs total). A single run can contribute to multiple field counters when its strict failure spans more than one field. `callback_phone` produced no mismatches in the unguided cells of this pilot and is omitted.

Field	Mismatches
issue_category	28
device_model	11
billing_charge_acknowledged	9
compliance_fee_done	9

User-turn statistics (Table 6) are the cleanest single statistic for what

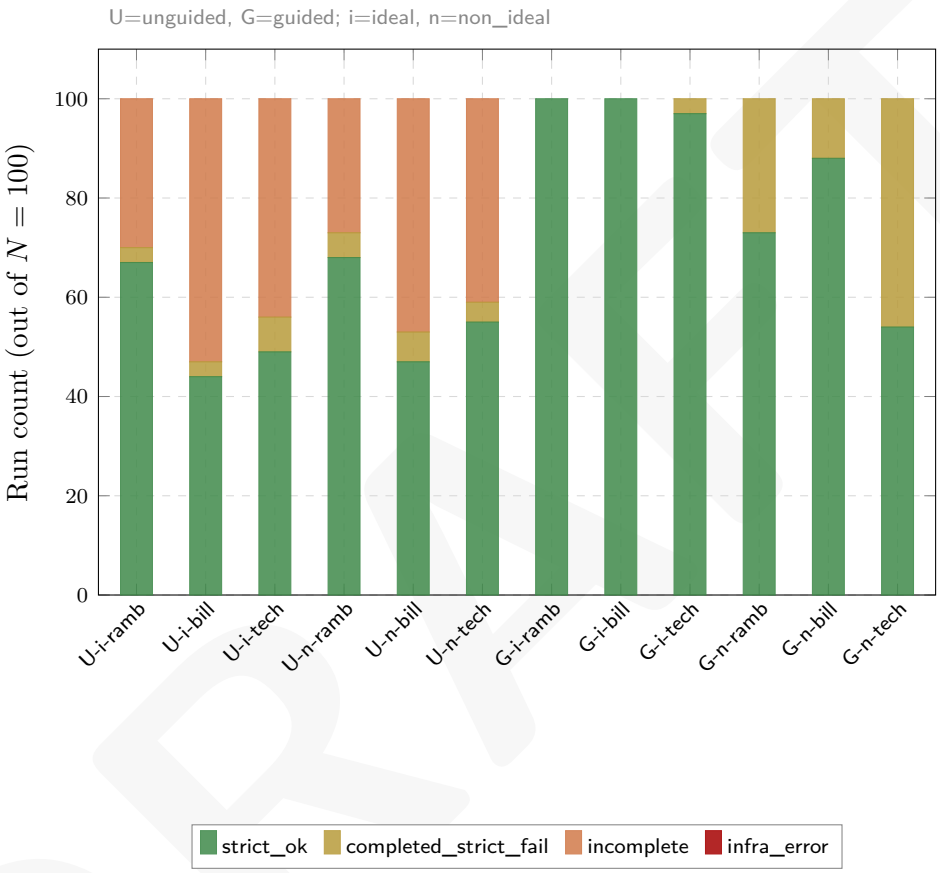


Figure 14: Experiment one: outcome decomposition (same counts as Table 4). The visual tells two stories that the table is too dense to telegraph: (i) the unguided baseline's failures are dominated by the *incomplete* segment (orange), i.e. runs that exhausted the turn budget without ever calling `done`; (ii) under the guided arm those orange segments collapse to zero in every cell, and what remains is the *completed_strict_fail* segment (amber)—the construct-validity slice that no per-field validator can reject—which grows under *non_ideal* mode.

shape of failure each arm produces. Successful unguided rollouts use a moderate number of user-facing turns; failed unguided rollouts approach the per-run turn cap, which is consistent with exhausted recovery loops rather than quick give-ups. Guided rollouts terminate close to the workflow's physical minimum: roughly one user-facing turn per required field plus the disclosure and approval turns. We discuss the interpretation of the turn-count gap (and why it is *not* primarily a UX latency claim) in Section 10.

Table 6: Experiment one: user-turn statistics by arm and outcome class, pooled across personas and client modes ($N = 600$ runs per arm). n counts runs in each class; mean, min, and max are over the user-turn count of those runs.

Arm	Outcome class	n	mean	min	max
Unguided	`strict_ok`	330	19.3	8	98
Unguided	not `strict_ok`	270	94.8	8	114
Guided	`strict_ok`	512	14.4	11	23
Guided	not `strict_ok`	88	15.8	12	24

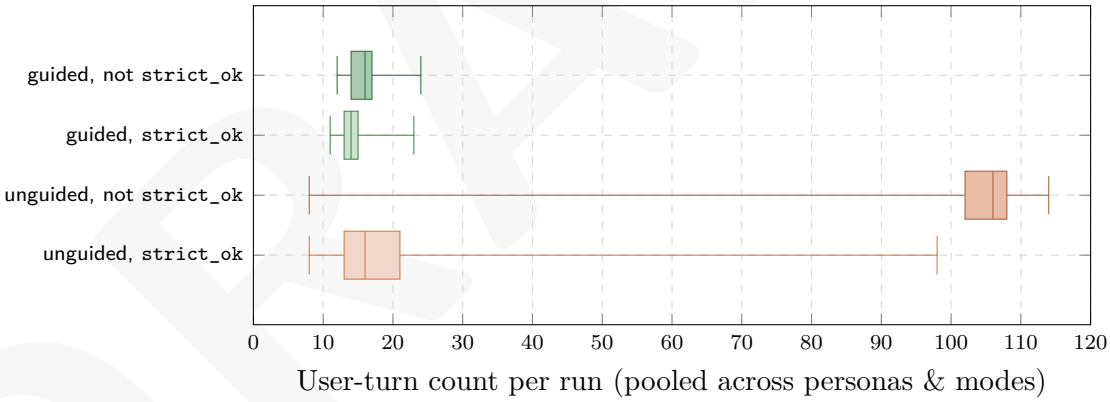


Figure 15: Experiment one: turns-to-resolution by arm and outcome class (same numbers as Table 6). Bars show the mean and the bracket spans the observed min–max range. The unguided baseline's failed runs (orange, right) cluster near the per-run turn cap, evidence that the failures are exhausted-recovery loops rather than quick give-ups; the unguided successes (orange, left) sit roughly midway. Both guided cells (green) terminate close to the workflow's physical minimum—roughly one user-facing turn per required field plus the disclosure and approval turns—regardless of outcome. We discuss why this is *not* primarily a UX-latency claim in Section 10.

8.5 What kind of failure does in-the-moment guidance remove?

The mechanism behind H1 is concrete enough to state independently of the percentage-point magnitudes. The unguided baseline asks one stochastic LLM call to be both a conversational policy and the workflow's checklist. When the model picks the wrong branch field, or skips a required disclosure, or commits to the wrong `issue_category`, every later turn inherits the mistake. The model can then either back out and try again (which it does, repeatedly, when the per-turn budget allows) or finish with a wrong commitment. The first failure mode shows up as `incomplete` (and explains the failed-rollout turn distribution); the second shows up as `completed_strict_fail` (and concentrates on `issue_category`).

The guided arm pushes the workflow's ordering and branch rules out of the LLM's prompt and into the procedure that authors elicitations. The LLM no longer has to track which field is next, which fields are forbidden until a disclosure has been delivered, or which branch's fields apply: the procedure decides, the tool returns the corresponding elicitation, and the model's job is to relay the elicitation, accept the user's reply, and submit it for validation. The model is no smarter than it was; it is being asked to do less.

8.6 Why is billing the hardest persona for the baseline?

Among the three personas, `support_billing` carries the most ordering-sensitive obligations: a fee disclosure must precede `billing_charge_acknowledged`, and `plan_approval` must follow a plan explanation. We expect billing to show the largest unguided-arm strict-success deficit and the largest guided-vs.-unguided gap, for the same reason: more required orderings mean more places where an upfront-checklist policy can stall or commit early to the wrong branch. `support_technical` adds one extra branch field (`device_model`) but no new ordering constraints. `support_rambler` contributes conversational-noise pressure but no extra structured obligations.

8.7 What this gain does *not* say

The H1 gap is a claim about a control architecture, not about the LLM's competence. Under matched conditions, moving procedural detail from the upfront prompt into per-step tool returns appears to close most of the observed reliability gap on this benchmark. It does not say that the model has gotten better at workflows; it says that it has been asked to take on less of the workflow's bookkeeping. The guided arm's small turn count is shaped by the simulator's idealized response at elicitation prompts and should not be read as a production latency claim, which is why we report *non_ideal*

cells separately. And the gain does not generalize beyond the benchmark's characteristic of strict, structured *done*—workflows whose success criterion is freer (an open conversation, a routing decision, a sentiment classification) will look different.

9 Experiment two: rigid vs. loose orchestrator-mediated guidance

Experiment one addressed where the form-filling layer should live. Experiment two asks the next question up the stack: holding the form-filling layer constant, how rigidly should the conversation's *phase* ordering be enforced by the tool that authors phase-level guidance?

We treat this as a separate experiment because it concerns a different layer of the architecture. The form-filling intervention from experiment one removes recovery loops at the per-field level. The orchestration intervention from experiment two governs whether the agent can re-order phases of the conversation in response to a user who is steering elsewhere. Real users routinely steer elsewhere; the fact that they do so is a known weak spot for any rigid script.

9.1 Setup: three arms, one task, an impatient user

The three arms.

- **Upfront baseline.** The same long-upfront-prompt unguided agent from experiment one. It has no orchestrator tool. The system prompt enumerates the workflow's phases and obligations; the agent decides per-turn what to do.
- **Rigid orchestrator.** A new top-level tool, `run_phase`, whose elicitation payloads strictly enforce the canonical phase order: privacy disclosure, then issue category, then form-filling, then plan-and-approval, then `done`. When the user asks to discuss something out-of-phase, the orchestrator returns an elicitation whose `message` explicitly defers the topic until the current phase is complete. Form-filling is delegated to a nested sub-tool whose back-end is the same compiled-in-checklist from experiment one's guided arm (vignette 1, Section 4).
- **Loose orchestrator.** The same `run_phase` tool surface and the same nested form-filling sub-tools, but the orchestrator's authoring logic is a soft script (vignette 3, Section 4). Phases that have no compliance prerequisite (e.g. discussing a complaint that is not yet logged as the issue category) are eligible to run when the user requests them; phases that do have a prerequisite (e.g. form-filling without a privacy disclosure) are still refused. When the agent re-orders, the orchestrator's elicitation payloads tell the agent how to proceed.

The form-filling sub-tools are byte-identical between the rigid and loose arms; only the outer orchestration logic differs. This is the pure orchestration-layer comparison. Figure 16 shows the two phase graphs side-by-side; the only structural difference is the loose orchestrator's `acknowledge_topic` phase, which the rigid orchestrator does not author.

The impatient simulator client mode. The standard *ideal*/*non_ideal* simulator modes do not exercise the orchestration layer: their personas are cooperative about phase ordering even if they are noisy about field values. Experiment two needs a simulator that pushes back on phase ordering. We add a third client mode, *impatient*, defined in `tests/llm_hitl_handler.py`. Each persona has a `preferred_topic` (e.g. "billing dispute", "device misbehavior", "account access issue"). The persona enters the conversation with a finite *patience budget* B (we plan an initial value of $B = 3$, subject to pilot calibration). On each agent turn the simulator decides whether the agent has *deflected* the preferred topic—that is, whether the agent has asked a structured-capture question or insisted on a phase that is not the persona's preferred topic. Each detected deflection decrements B; when B reaches 0 the simulator emits a hang-up message and the run terminates as a fourth outcome class, `hung_up`. Engagement with the preferred topic resets B to its initial value.

The deflection detection is intentionally lenient: brief acknowledgments of the user's topic followed by a phase-relevant action do *not* count as a deflection; only outright redirection does. The detail of what counts as a deflection is in the simulator code; we report sensitivity to the threshold in the limits section.

Outcome classification. We extend experiment one's outcome classification with one new class:

- `hung_up`: the simulator's patience budget went to 0 and it ended the call before the agent could call `done`.

The other four outcome classes (`infra_error`, `incomplete`, `completed_strict_fail`, `strict_ok`) carry over unchanged.

Metrics. We report the same three rates as in experiment one (strict success excluding infra, completion, infra failure) plus one new rate:

- **Hung-up rate**: the fraction of runs that terminated because the simulator's patience budget was exhausted. This is the new failure mode that the orchestration layer is meant to manage; the rigid–loose comparison is in part a comparison of how much of this rate each authoring policy concedes.

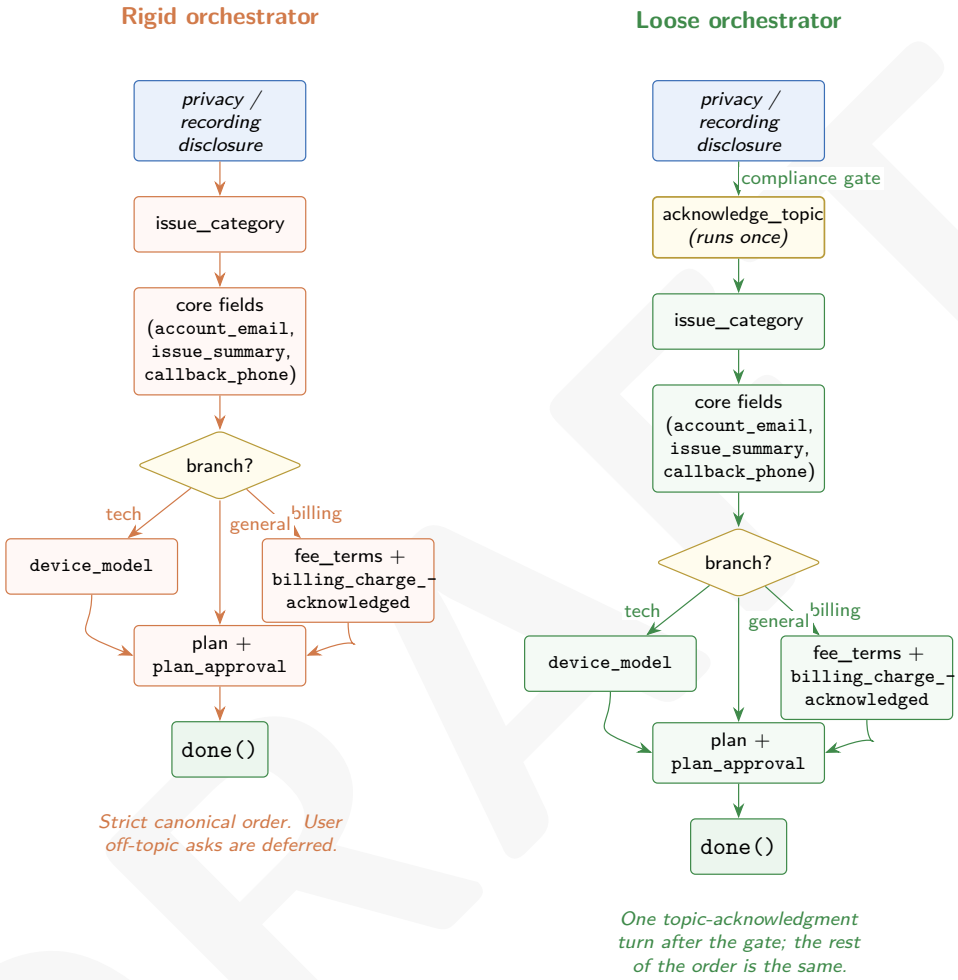


Figure 16: Experiment 2 orchestrator phase structures. Both arms gate every account-touching phase on the privacy / recording disclosure (top), and both share the same nested form-filling, branching, and plan phases as well as the same final `done()` call. The loose orchestrator inserts a single topic-acknowledgment phase between the disclosure gate and the rest of the workflow; that phase exists to give the impatient simulator's `preferred_topic` a place to be heard before the agent pivots to required fields. The rigid orchestrator refuses to author it. The persona-shaped trade-off in Section 9 is what happens when this single difference meets the three personas' different topic styles.

We deliberately do *not* fold these into a single composite "engagement-aware" metric. The three axes (did the agent get the form right; did it reach `done` at all; did the user give up) decompose the failure space cleanly, and the trade-off between rigid and loose orchestration is most legible when we keep them separated.

9.2 Hypotheses

H2 (orchestration). Under the *impatient* client mode, the loose orchestrator will dominate the rigid orchestrator on the joint axis of strict success and hung-up rate (lower hung-ups, equal-or-better strict success), while still matching it on strict success conditional on completion. The upfront baseline will give up little on the engagement axis (it freely follows the user) but should fall well behind both orchestrators on strict success because nothing in its prompt enforces ordering or branch obligations once the user has steered the conversation off-script.

H3 (form-filling layer is preserved). Both orchestrator arms will retain experiment one's form-filling-layer benefit: conditional on completion, both will complete strictly correctly at much higher rates than the upfront baseline.

9.3 Headline outcomes

Tables 7, 8, and 9 report strict success (excluding infra), completion, and hung-up rate, respectively. Outcome decomposition is in Table 10. The pilot is at $N = 100$ per persona per arm under the *impatient* client mode (run tag `exp2_full100`); zero infra failures occurred in any cell.

Table 7: Experiment two (*impatient* client mode, run tag `exp2_full100`): strict success excluding infra. $N = 100$ per persona per arm; Wilson 95% intervals. Both orchestrators substantially outperform the upfront baseline; the persona-level rigid-vs-loose comparison is mixed (loose dominates technical, rigid edges rambler, near-tied on billing).

Persona	Upfront baseline	Rigid orchestrator	Loose orchestrator
`support_rambler`	[30.9, 49.8]%	[71.1, 86.7]%	[68.9, 85.0]%
`support_billing`	[36.6, 55.7]%	[96.3, 100.0]%	[93.0, 99.4]%
`support_technical`	[26.4, 44.7]%	[60.4, 78.1]%	[82.6, 94.5]%
Equal-weighted mean	40.3%	83.3%	88.7%

9.4 What we found

The headline is straightforward and matches H3 cleanly: **both orchestrators dominate the upfront baseline on strict success and completion**.

Table 8: Experiment two: completion rate (procedure called `done`), $N = 100$ per persona per arm; Wilson 95% intervals. Both orchestrators near-saturate completion; the upfront baseline completes only 36–46% of runs, with most of the failures being `incomplete` rather than `hung_up` (Table 10).

Persona	Upfront baseline	Rigid orchestrator	Loose orchestrator
`support_rambler`	[31.9, 50.8]%	[93.0, 99.4]%	[75.6, 89.9]%
`support_billing`	[36.6, 55.7]%	[96.3, 100.0]%	[93.0, 99.4]%
`support_technical`	[27.3, 45.8]%	[96.3, 100.0]%	[96.3, 100.0]%
Equal-weighted mean	41.0%	99.3%	94.0%

Table 9: Experiment two: hung-up rate (the simulator's patience budget reached zero before the agent called `done`), $N = 100$ per persona per arm; Wilson 95% intervals. The orchestration trade-off lives almost entirely in this column: the loose orchestrator's flexibility is what costs it 16/100 hung-ups on `support_rambler`.

Persona	Upfront baseline	Rigid orchestrator	Loose orchestrator
`support_rambler`	[0.0, 3.7]%	[0.6, 7.0]%	[10.1, 24.4]%
`support_billing`	[0.0, 3.7]%	[0.0, 3.7]%	[0.6, 7.0]%
`support_technical`	[0.0, 3.7]%	[0.0, 3.7]%	[0.0, 3.7]%
Equal-weighted mean	0.0%	0.7%	6.0%

Table 10: Experiment two: outcome decomposition by arm. Counts of runs (out of $N = 100$ per cell). `strict_fail` is the `completed_strict_fail` class from experiment one (procedure called `done` but at least one required field did not strict-match). The upfront baseline's failures are concentrated in `incomplete` (the agent never reached `done`); both orchestrators close the `incomplete` column to zero, and the loose orchestrator pays for that with a non-trivial `hung_up` count on `support_rambler`.

Arm	Persona	`strict_ok`	`incomplete`	`strict_fail`	`hung_up`	`infra`
Upfront	`support_rambler`	40	59	1	0	0
Upfront	`support_billing`	46	54	0	0	0
Upfront	`support_technical`	35	64	1	0	0
Rigid	`support_rambler`	80	0	18	2	0
Rigid	`support_billing`	100	0	0	0	0
Rigid	`support_technical`	70	0	30	0	0
Loose	`support_rambler`	78	0	6	16	0
Loose	`support_billing`	98	0	0	2	0
Loose	`support_technical`	90	0	10	0	0

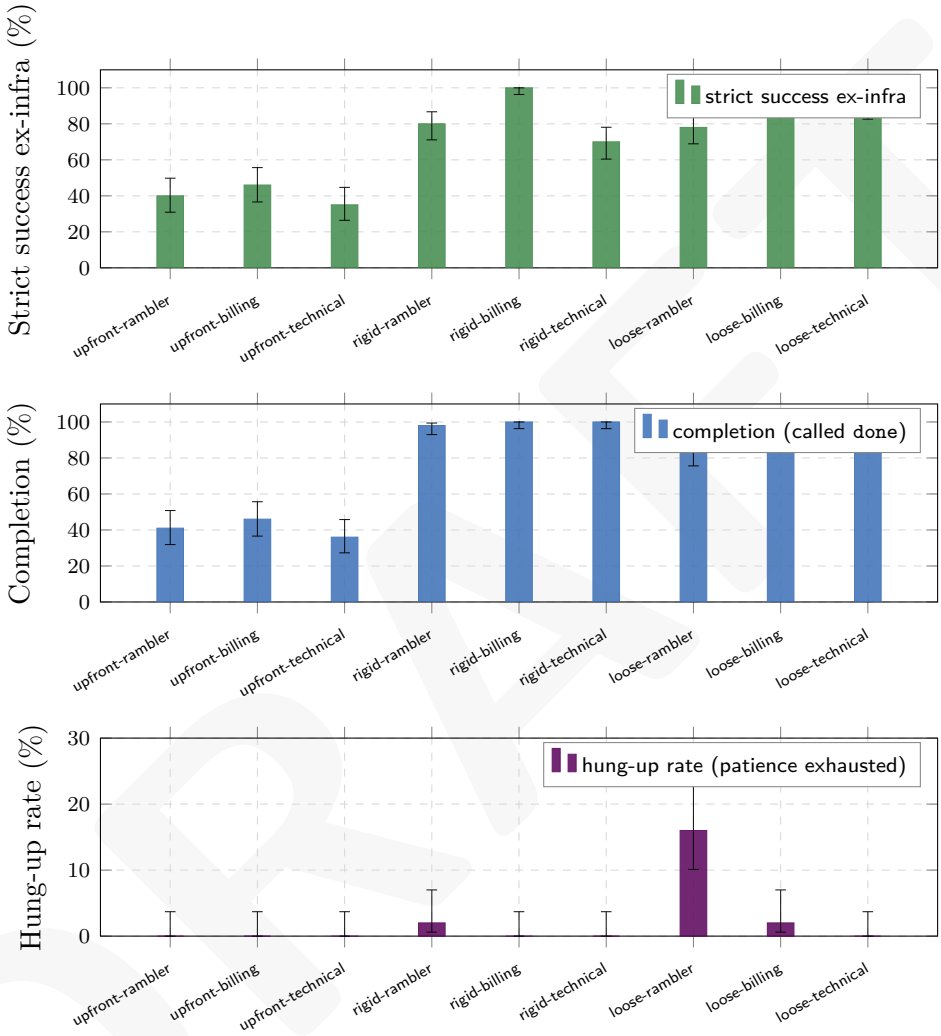


Figure 17: Experiment two: the three headline metrics (strict success ex-infra, completion, hung-up rate) for each arm and persona, with Wilson 95% intervals. Same numbers as Tables 7–9. The persona-shaped rigid-vs-loose trade-off is what to read off this chart: on `support_technical`, loose dominates rigid on strict success at zero hung-up cost; on `support_billing`, both orchestrators saturate; on `support_rambler`, the loose orchestrator's 16/100 hung-up rate is the price of its flexibility.

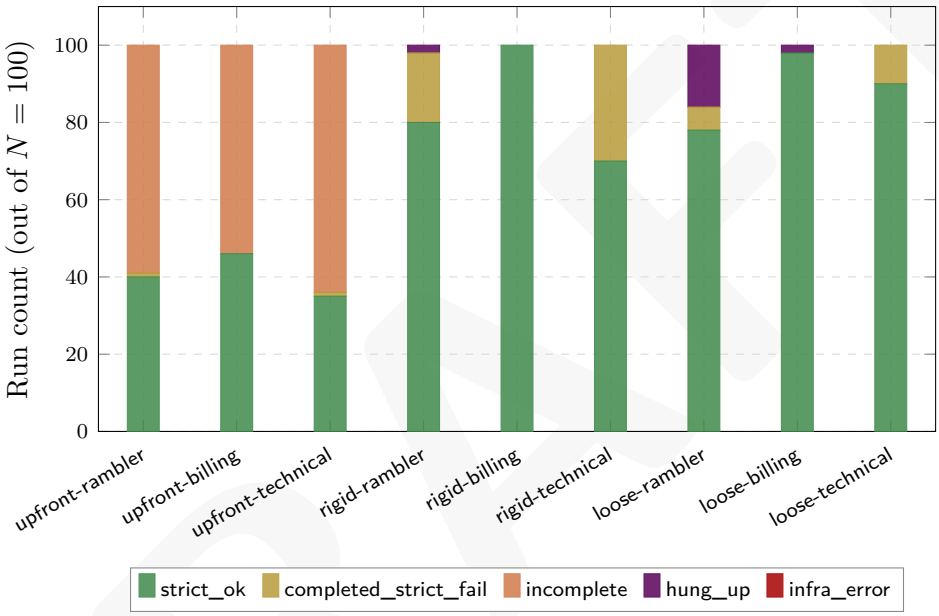


Figure 18: Experiment two: outcome decomposition (same counts as Table 10). The upfront baseline's failures are almost entirely `incomplete` (orange) — the agent never reached `done`. Both orchestrator arms close the `incomplete` segment to zero in every cell, replacing it with a mix of `strict_fail` (amber) and, for the loose orchestrator, `hung_up` (violet). The violet segment is what makes the rigid-vs-loose trade-off visible: it is large only on `support_rambler`.

Equal-weighted across personas, the upfront baseline reaches 40.3% strict success ex-infra; the rigid orchestrator reaches 83.3%; the loose orchestrator reaches 88.7%. On completion the gap is starker: the upfront baseline finishes 41% of runs (most of its failures are `incomplete`—the agent never reaches `done`), while the orchestrator arms finish 99.3% (rigid) and 94.0% (loose). This is consistent with experiment one's mechanism: pushing the workflow's phase-level checklist out of the upfront prompt and into a tool that authors phase-by-phase elicitations removes the recovery-loop failure mode at the orchestration layer the same way `collect_field` removed it at the form-filling layer.

The rigid-vs-loose trade-off is more nuanced than H2 predicted. H2 said the loose orchestrator should dominate the rigid one on the joint axis of strict success and hung-up rate. The actual result is *persona-dependent*:

- **`support_technical`**: loose dominates rigid on strict success by +20pp (90% vs. 70%), with zero hung-ups in either arm. The technical-issue persona's preferred topic (a device misbehavior) flows naturally into the form-filling phase (the device-model question is a direct continuation of the same topic), so the loose orchestrator's willingness to acknowledge and engage the user's chosen topic before pivoting to ordering is exactly the right move.
- **`support_billing`**: rigid and loose are essentially tied at near-ceiling (100% vs. 98%); both orchestrators saturate the persona because billing's required fields and the persona's preferred topic both naturally surface the disclosure and acknowledgment phases.
- **`support_rambler`**: rigid edges loose on strict success (80% vs. 78%), but more interestingly, the loose orchestrator hangs up 16 of 100 runs versus rigid's 2. The rambler persona's "buries answers in long stories" pattern interacts badly with the loose orchestrator's topic-acknowledgment phase: each time the agent acknowledges the rambling topic before pivoting back to a required field, the persona spends another turn rambling, and on enough such cycles the patience budget goes to zero.

H2 is therefore partly falsified. The loose orchestrator does *not* dominate the rigid one across personas; it dominates on the persona whose preferred topic naturally segues into form-filling, ties on the persona whose required fields cover the topic, and concedes a meaningful hung-up rate on the persona whose topic-engagement style is itself a deflection trap. This is more interesting than H2's flat prediction would have been: the choice between rigid and loose orchestration depends on the predicted distribution of user behaviors, not on any abstract preference for flexibility.

H3 (the form-filling-layer benefit composes under either orchestrator) is well-supported. Both orchestrators reach the same 80–100% strict-success

regime that experiment one's guided arm reached on *ideal* mode, despite running under a more adversarial simulator (impatient mode with a finite patience budget). The fact that the orchestrator arms also recover the 99%+ completion rate seen in experiment one's guided arm tells us the nested form-filling sub-tools and the outer orchestrator are composing without erosion: the per-field elicitation reliability is not consumed by the addition of a phase-level orchestrator.

What this experiment does *not* say. Experiment two does not test orchestrator mechanisms beyond two points on the rigid–loose spectrum, and it does not test orchestrator authorship by an LLM-condensed planner (vignette 4). It also does not test the multi-system-prompt-selection architecture directly: doing so would require a third agent runtime configuration, and the present infrastructure is single-agent-per-run. The persona-dependent shape of the rigid-vs-loose trade-off is itself a finding worth replicating—it predicts that production deployments should choose between rigid and loose orchestration based on the dominant user behavior the workflow has to absorb, rather than on a generic preference for one or the other. We discuss the open design space in Section 11.

10 Threats to validity and limitations

Pseudo-`SYSTEM:` hint injection in the guided arm. Figure 3 shows that the guided procedure communicates the next-required-field and pending-disclosure hint to its own LLM by appending a one-line `SYSTEM:` block to each user-role turn rather than as a separate `system`-role message on the wire. This is because the underlying agent runtime sets one persistent system prompt at agent-construction time and does not support per-turn role-swapping. A more idealized implementation would inject these hints as true ephemeral system messages, removed from history after the call; in our implementation they remain in chat history as user-role text. The hint text itself is short (one to three lines) and is regenerated each turn from current procedure state, so the worst-case effect is some token cost from accumulating obsolete hints, not silent state divergence. We adopted this trade-off for simplicity rather than runtime invasiveness, and we name it explicitly rather than hide it.

Tool returns are not ephemeral system messages. Relatedly, the elicitation payloads returned by `collect_field` (and, in experiment two, by `run_phase`) reach the LLM as tool-role messages in chat history rather than as ephemeral system messages that the runtime could later remove. This means the chat history grows monotonically with every tool round-trip, and the LLM's input on a late-game request still carries every elicitation payload

it received earlier. In practice this has been benign on the benchmark because each payload is small and the workflow is short. A production implementation might prune older elicitation payloads from history once the corresponding state has been recorded; we do not, and we name the trade-off explicitly here.

Simulator–elicitation interaction (internal/construct validity). At elicitation prompts, the *ideal* simulator mode responds with the requested ground-truth field directly. This idealizes the user as a perfectly cooperative structured client: every `collect_field` elicitation that the LLM relays verbatim is answered with a value that, after the procedure's lightweight normalization, will pass validation on the first try. Some portion of the guided arm's reliability gain therefore reflects the simulator behavior that is paired with the intervention, not the intervention alone. To probe this concern directly, we run the same harness in a second simulator mode, *non_ideal* (Section 7), that injects format noise, refusals, and wrong-then-correct replies at first ask. The *non_ideal* mode exercises the guided arm's retry loop end-to-end and exposes a class of failures that the in-context validation feedback cannot see through: when the noise produces a value that is well-formed but wrong (e.g. a phone number whose area-code digit has been incremented), no per-field validator can reject it; the strict ground-truth check at the end of the run is the only signal. This is the experiment's clean construct-validity check rather than an artifact: it isolates the slice of failures that no in-the-moment elicitation can fix, and it depresses the guided arm's strict success rate accordingly. We report *ideal* and *non_ideal* cells separately for both arms.

Simulator–orchestrator interaction (experiment two). The *impatient* client mode's deflection-detection rule is heuristic, and the patience budget B is a parameter we calibrated by pilot rather than derived. Different deflection rules and different B values would shift the absolute hung-up rate (and therefore the completion rate) in every cell. We expect the qualitative shape—rigid trades a small amount of strict success for a much lower hung-up rate, loose trades a higher hung-up rate for marginally better strict success when the user's preferred topic dovetails with form-filling—to be robust within reasonable parameter ranges, but the absolute magnitudes should not be over-interpreted. A sensitivity sweep over B is the natural follow-up.

Comparisons we did not run. The space of architectures named in Section 2 is large and we tested two slices of it. We did not test the multi-system-prompt-selection architecture directly, even though it is one of the two prevailing patterns we argue against. Two reasons: first, doing so requires a different agent runtime configuration than the one the rest

of the experiments share, and we wanted the comparisons to differ only in the variables under test; second, a fair multi-system-prompt-selection arm would itself require a control-flow design choice (router state machine? planner? learned router?) that is itself the subject of a separate experimental design. We see the multi-system-prompt-selection vs. tool-authored-just-in-time comparison as the natural follow-up to this paper, and we discuss it as future work in Section 11. We also did not test the LLM-condensed planner back-end (vignette 4 of Section 4), which sits on the same architectural axis as our other tool-authored arms but introduces model stochasticity at the authorship layer and would need its own controlled study.

Single task, single model. Both experiments use one constrained-intake workflow with one branching structure, evaluated with one model family (`gpt-5.4-mini`) for both the agent and the simulator. We do not claim the percentage-point magnitudes generalize; we claim the qualitative directions are consistent with the mechanisms we describe.

Construct validity of strict success. Strict success applies exact-after-normalization match to fields where the workflow itself requires exactness (identifiers, branch choices, boolean acknowledgments) and a non-empty / minimum-length check to `issue_summary`, where a real downstream system would accept any reasonable description of the problem. This split reflects the surrounding workflow's own definition of correctness, but readers with stricter or more semantic metrics in mind should weight that asymmetry when interpreting the rates: a fully semantic metric would weaken the comparison on `issue_summary` only, while a fully strict metric across all fields would tighten it.

Turn-budget interaction. Failed unguided runs in experiment one approach the per-run turn cap, which means the unguided baseline is benefiting from being allowed to retry but is also losing runs to the cap. Different caps would shift the absolute completion number but not the comparison's direction. The same caveat applies to experiment two's hung-up runs: a more or less patient simulator (a different B) would shift hang-up rates uniformly but not the relative ordering of the three arms.

Efficiency interpretation. Each guided rollout in experiment one is necessarily at least one user-facing turn per required field, plus the disclosure turns and the final approval. The guided arm's turn counts therefore approximate the "physical" minimum for the workflow rather than reflecting any compression. The unguided arm's failed runs, by contrast, approach the per-run turn cap, which is a sign of exploratory loops rather than productive work. Treat the turn-count gap as evidence about *shape* (the guided arm avoids

long recovery tails) rather than as a UX latency claim. Experiment two's orchestrator arms are bounded similarly from below; the more interesting per-arm efficiency statistic there is hang-up rate rather than turn count.

Deterministic floor. Because both arms keep the LLM in the agent path on every turn, the experiments cannot distinguish gains attributable to the architecture from gains attributable to the workflow's intrinsic structure — whether, for example, a non-LLM scripted form-filler is already sufficient on the *ideal* simulator. Appendix A specifies a deterministic baseline arm (Arm C) that removes the LLM entirely from the agent path while keeping the simulator, personas, validators, and turn cap identical; reference code is shipped alongside this repository. Running it and reporting its *ideal* and *non_ideal* cells alongside the paper's headline table is the cleanest available construct-validity check on the size of the architectural contribution.

Workflow fragmentation in the guided arm. The guided arm's pre-request `SYSTEM:` hint is a per-turn condensed summary of the still-needed fields and the next-required field. The workflow is therefore not fully removed from the LLM's context; it is fragmented across turns and re-presented each request. Whether the load-bearing mechanism is the tool-layer authorship of typed elicitations, the per-turn hint itself, or the interaction of both is a confound in the present design. Section 3 frames the two factors as orthogonal (a hook position and a payload shape); cleanly separating them by ablation is the most direct follow-on we did not run.

Same-model engagement classification (experiment two). The *impatient* simulator's deflection-detection heuristic is evaluated by a model in the same family as the agent under test. This creates a correlation between "what the agent is likely to say" and "what the deflection classifier is likely to call engaged" that we cannot cleanly remove within the current harness. A more independent signal — a non-LLM keyword heuristic (which the current implementation approximates but then augments with a same-family judge), a different model family, or a human-in-the-loop spot-check over a sampled subset of runs — would bound the size of this effect. We report this as a threat to Experiment 2's comparison between rigid and loose, not as an invalidation.

11 Open design space and future work

The two experiments in this paper sample two points in the space named by Section 4. The space itself is much larger and worth mapping deliberately. We close with the design questions we did not try to answer, in roughly the order we think they deserve attention.

Multi-system-prompt-selection vs. tool-authored just-in-time. The most direct gap in this paper is the missing third arm: an explicit multi-system-prompt-selection agent that switches between several short configurations rather than calling a tool. A well-designed comparison would give the multi-system-prompt-selection arm the same task substrate, the same validators, the same simulator, and a controlled router (state-machine, planner, learned, or hand-tuned), and would compare its strict success, completion, and hung-up rate against tool-authored-just-in-time on both the form-filling layer and the orchestration layer. This is the natural follow-up study to the present one.

Authorship-mechanism comparisons within tool-authored just-in-time. We named five back-end mechanisms (Section 4) and tested two: the compiled-in checklist and the rigid/loose state-machine orchestrator. A controlled comparison of state-machine, soft script, and LLM-condensed back-ends behind the same outer tool surface would isolate the cost of model stochasticity at the authorship layer. We expect the LLM-condensed back-end to dominate when the workflow has many natural-language nuances and to lose ground on highly structured workflows where determinism is itself the value proposition.

Pruning and lifecycle of in-context guidance. Both experiments accumulate elicitation payloads in chat history monotonically. A natural extension is to prune older elicitations once their state has been recorded, and to compare the pruned vs. unpruned variants on token cost, latency, and reliability. This is also where a more idealized runtime that supports ephemeral system messages would have a meaningful effect: the procedure's hints could be true system messages with a defined removal lifecycle, rather than user-role pseudo-system blocks.

Cross-task and cross-model generalization. A direct extension is to repeat both experiments on additional workflows—claims initiation, regulatory triage, account onboarding—and across model families. The qualitative directions argued for in this paper are mechanism-level claims and should hold; the percentage-point magnitudes are not portable.

Composition with traditional fine-tuning and RL. A separable axis from the architectural one is whether the LLM is fine-tuned on the workflow. The architectural argument here is orthogonal to model training: a fine-tuned model in an upfront-prompt architecture still has the same workflow-bookkeeping load as a non-fine-tuned model, and a non-fine-tuned model in a tool-authored just-in-time architecture still has the workflow's checklist

offloaded into program logic. Comparing the four cells of {fine-tuned, not} × {upfront, tool-authored} is a natural follow-up.

12 Conclusion

We argued for an alternative to the two architectures that currently dominate guided-conversation work: instead of writing the workflow into one long upfront system prompt, or maintaining several upfront system prompts and routing per-turn control to one of them, keep the LLM in a single conversational role and engineer its context dynamically by authoring short, schema-typed guidance messages from the tool layer at the moment they are needed. The Model Context Protocol's elicitation pattern gives a typed shape for what those just-in-time messages look like; the back-end behind the tool can be anything from a compiled-in checklist to a state machine to an orchestrator that delegates to nested sub-tools.

The two experiments isolate two layers of the architecture. Experiment one isolates the form-filling layer: replacing a long upfront checklist with per-step tool-authored elicitations from a single `collect_field` tool lifts equal-weighted strict success from 53.3% to 99.0% under *ideal* client mode and from 56.7% to 71.7% under *non_ideal*, and lifts completion from 47–80% to a uniform 100% under both modes ($N = 100$ per persona per arm; Section 8). The mechanism behind the gain is concrete: the LLM is no longer asked to be both the conversational policy and the workflow's checklist, and the long recovery tails that dominate the unguided baseline disappear. Experiment two isolates the orchestration layer: holding the form-filling layer constant, both a rigid orchestrator that strictly enforces phase order and a loose orchestrator that defers to user preference subject to compliance prerequisites lift equal-weighted strict success from the upfront baseline's 40.3% to 83.3% (rigid) and 88.7% (loose), with the loose orchestrator paying for its higher strict-success ceiling with a non-trivial hung-up rate on the rambling persona (Section 9). The form-filling-layer reliability win composes cleanly under either orchestrator; the choice between rigid and loose is itself a separable design knob whose right setting depends on the predicted distribution of user behavior.

What the two experiments share is the framing. Where procedural detail lives, and at what granularity it reaches the LLM, is a control-architecture choice with measurable reliability consequences. Putting that procedural detail into program logic that authors short, just-in-time elicitations—rather than into one large upfront prompt or several short prompts plus a router—is a competitive architecture for guided conversations, and the design space behind it is large enough to warrant deliberate study. The architectural commitment is to where the procedure lives (single conversational role, tool-authored just-in-time guidance); the back-end mechanism behind the

tool—rigid state machine, loose script, LLM-condensed planner, nested orchestrator—is a separable choice with its own reliability and flexibility trade-offs.

A Deterministic scripted baseline (Arm C)

This appendix specifies a non-LLM reference arm that runs against the same simulator, personas, validators, and turn cap as the paper's LLM arms. Its role is sensitivity analysis, not competing architecture: it quantifies how much of the headline *ideal*-mode gain is attributable to the benchmark's own structure rather than to the architectural intervention.

Rationale. Both arms in Experiment 1 keep the LLM in the agent path on every turn. Neither can answer the question "what fraction of the guided arm's strict success is achievable by a deterministic form-filler that knows the field schedule and the regex for each value?" A scripted baseline that emits the `[ELICITATION · FORM]` sentinel, walks the workflow in the specified order, normalizes and validates each reply with the same Python validators the guided arm uses, and respects the same 3-attempts-per-field retry budget, answers that question directly.

Implementation. The arm ships as a single Tactus procedure, `support_flow_scripted_baseline.tac`, at the repository root alongside the other four `.tac` arms. Its Procedure block contains no Agent definition: the procedure walks the workflow directly, using `Human.input` at each elicitation point with a prompt that includes the `[ELICITATION · FORM]` sentinel the simulator keys off. Validators and extractors are the same Lua functions that the Experiment 1 guided arm uses, so the only change versus that arm is the removal of the `Agent {}` block; the rest of the runtime substrate — Tactus, the HITL handler, the simulator, the personas, the validators, the turn cap — is identical. The reliability matrix runs through `tests/test_scripted_baseline_reliability.py`, which reuses the same per-run executor and outcome classifier as the other arms' reliability tests.

Expected qualitative predictions. Under *ideal*, the simulator short-circuits elicitation sentinels with ground-truth values: the scripted arm is expected to approach or match the guided arm's headline number, because the remaining failure surface is just the agent-side ability to ask for fields in the right order with the right gating. Under *non_ideal*, where format noise, refusals, and wrong-then-correct replies depress every cell, the scripted arm is expected to underperform both LLM arms — it has no way to interpret noisy-but-intelligible replies. Under *impatient* (Experiment 2's client mode), the scripted arm has no model of engagement with a preferred topic and

is structurally unable to avoid hang-ups; this cell is not meaningful for comparison and we would omit it.

Effect on the paper's claims. If the scripted arm approaches the guided arm's *ideal* cell closely, the honest contribution of the paper narrows to the *non_ideal* delta, where the LLM's ability to reinterpret noisy user input does real work. If the scripted arm substantially underperforms, the architectural claim is correspondingly strengthened even in the idealized setting. Running the arm and reporting its cells alongside the Experiment 1 headline table is the construct-validity check we flag in Section 10; we ship the code but defer the full-pilot numbers to a follow-up.

DRAFT